



PROJECT AKHIR KELOMPOK D
PAS 241302-ALGORITMA DAN PEMROGRAMAN

Dosen Pengampu:

Lutfiah Maharani Siniwi, S.Stat., M.Stat.

NIP. 199903272025062013

Disusun Oleh :

Muhammad Nizar Aflah (K1D025007)

Danish Prasetya (K1D025011)

Erga Selviana Dewi (K1D025018)

Aurella Kristy Sitepu (K1D025021)

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI

UNIVERSITAS JENDERAL SOEDIRMAN

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI STATISTIKA

2026

DAFTAR ISI

DAFTAR ISI.....	i
BAB I PENDAHULUAN.....	1
1.1. Latar Belakang	1
1.2. Tujuan Project	1
1.3. Ruang Lingkup.....	2
BAB II ANALISIS MASALAH.....	3
2.1. Uraian Soal dan Batasan Masalah.....	3
2.2. Tabel Input-Proses-Output	4
2.3. Pembagian Tugas Kelompok	5
BAB III PSEUDOCODE DAN FLOWCHART	6
3.1. Program 1	6
3.2. Program 2	7
3.3. Program 3	8
3.4. Program 4.....	10
3.5. Program 5	13
3.6. Program 6.....	15
BAB IV IMPLEMENTASI PROGRAM.....	18
4.1. Program 1	18
4.2. Program 2	19
4.3. Program 3	20
4.4. Program 4.....	22
4.5. Program 5	23
4.6. Program 6.....	26
BAB V PENJELASAN LINE-BY-LINE-CODE	29
5.1. Program 1	29
5.2. Program 2	30
5.3. Program 3	31
5.4. Program 4.....	33
5.5. Program 5	35
5.6. Program 6.....	36
BAB VI PENGUJIAN PROGRAM	39

6.1. Tabel Pengujian.....	39
6.2. Screenshot Hasil Running.....	43
6.3. Interpretasi Output	47
BAB VII PENUTUP.....	49
7.1. Kesimpulan.....	49
7.2. Kendala yang Dihadapi	49
7.3. Saran Pengembangan	49
LAMPIRAN	51

BAB I

PENDAHULUAN

1.1. Latar Belakang

Pemahaman algoritma merupakan dasar utama dalam memecahkan masalah komputasional. Laporan ini disusun sebagai pemenuhan Tugas Project Akhir mata kuliah Algoritma dan Pemrograman. Project ini melatih penerapan logika ke dalam dua bahasa pemrograman, yaitu Python dan R, yang sangat relevan untuk kebutuhan analisis data.

Di era digital saat ini, kemampuan untuk mengubah konsep algoritmik menjadi implementasi kode yang efisien dan benar telah menjadi keahlian penting bagi mahasiswa yang berkecimpung dalam bidang teknologi informasi, ilmu data, maupun berbagai disiplin ilmu lainnya yang memanfaatkan pemrosesan komputasi. Setiap bahasa pemrograman memiliki struktur dan cara kerja yang berbeda, sehingga memahami perbandingan antara keduanya dapat memperdalam pemahaman tentang prinsip dasar pemrograman itu sendiri. Project ini fokus pada enam kasus permasalahan yang representatif, mencakup berbagai aspek komputasional mulai dari pengolahan teks dan string, perhitungan matematis akar persamaan kuadrat, pengolahan data identitas seperti NIP ASN, klasifikasi data berbasis cluster 3 dimensi, hingga perhitungan statistik interval konfidensi proporsi. Keberagaman kasus ini dirancang untuk menguji kemampuan mahasiswa dalam mengatasi berbagai jenis tantangan komputasional, mulai dari yang bersifat dasar hingga yang lebih kompleks yang melibatkan konsep matematis dan statistik.

Selain itu, pelaksanaan project secara berkelompok dengan menggunakan repository GitHub tidak hanya melatih kemampuan teknis, tetapi juga kerja sama tim dan manajemen proyek yang sesuai dengan praktik industri. Setiap anggota kelompok diharapkan tidak hanya mampu menulis kode, tetapi juga memahami logika di baliknya, melakukan validasi input, menangani kondisi khusus, serta mendokumentasikan hasil pengujian secara sistematis. Hal ini bertujuan untuk membentuk kompetensi yang komprehensif, sehingga mahasiswa siap menghadapi tantangan pemrograman dalam dunia kerja maupun penelitian selanjutnya.

1.2. Tujuan Project

Berdasarkan pedoman penugasan, project ini memiliki tujuan pembelajaran sebagai berikut:

1. Menganalisis permasalahan komputasional dengan menentukan input, proses, output, serta batasan masalah.
2. Menyusun algoritma dalam bentuk narasi, *pseudocode*, dan *flowchart* yang runtut.

3. Mengimplementasikan algoritma menggunakan bahasa R dan Python.
4. Membandingkan struktur logika program antara R dan Python.
5. Menerapkan variabel, tipe data, operator, *string*, percabangan, fungsi, *tuple/list/vector*, dan operasi matematika.
6. Melakukan validasi input dan menangani kondisi khusus pada program.
7. Bekerja sama dalam tim menggunakan repository GitHub dan membagi kontribusi secara adil.

1.3. Ruang Lingkup

Project ini mencakup perancangan dan pengujian enam program berbasis studi kasus: analisis teks, penggunaan *string*, akar persamaan kuadrat, pengolahan NIP ASN, klasifikasi *cluster* 3 dimensi, dan interval konfidensi proporsi. Setiap program dilengkapi dengan *pseudocode*, *flowchart*, serta penjelasan baris kode.

BAB II

ANALISIS MASALAH

2.1. Uraian Soal dan Batasan Masalah

Berikut ini adalah uraian keenam soal beserta dengan batasan masalahnya:

1. Program 1 – Analisis Teks

Program ini meminta pengguna memasukkan sebuah teks, lalu program menghitung jumlah kata dan jumlah kalimat di dalam teks tersebut. Batasannya, teks diasumsikan tidak mengandung tanda titik selain untuk mengakhiri kalimat, sehingga titik dapat dipakai sebagai penanda jumlah kalimat. Output yang dihasilkan adalah dua nilai: jumlah kata dan jumlah kalimat.

2. Program 2 – String

Program ini meminta membuat objek string K1 sampai K4 dalam Python dan R sehingga saat dicetak hasilnya sama persis dengan kalimat yang ditampilkan pada soal. Program ini menuntut ketelitian dalam penulisan karakter, terutama tanda kutip, spasi, dan tanda baca, karena hasil keluaran harus sama persis dengan format yang diminta. Batasan masalahnya adalah string yang dibuat tidak boleh berubah makna maupun susunan karakternya dari contoh yang diberikan.

3. Program 3 – Akar Kuadrat

Program ini membaca koefisien dari persamaan kuadrat $ax^2 + bx + c = 0$ lalu menentukan akar-akarnya. Program harus mampu membedakan apakah persamaan memiliki akar real atau hanya akar imajiner, lalu menampilkan akar real dengan tiga angka di belakang koma jika memang ada. Batasan masalahnya adalah proses penentuan hasil bergantung pada nilai diskriminan, sehingga program harus menangani kondisi ketika diskriminan bernilai negatif.

4. Program 4 – NIP ASN

Program ini membahas pengolahan NIP ASN untuk menampilkan tanggal lahir berdasarkan delapan digit pertama NIP. Delapan digit tersebut digunakan untuk mengambil informasi tahun, bulan, dan tanggal lahir, kemudian bulan diterjemahkan ke nama bulan yang sesuai melalui percabangan. Batasan masalahnya adalah format NIP harus sesuai dengan ketentuan, karena program hanya membaca delapan digit awal sebagai data lahir ASN.

5. Program 5 – Cluster 3D

Program ini berisi klasifikasi titik dalam ruang tiga dimensi ke dalam cluster A, B, atau C berdasarkan jarak terdekat dari titik pusat masing-masing cluster. Program

menerima koordinat sebuah titik U dalam bentuk tuple tiga elemen, lalu menghitung jarak ke ketiga pusat cluster dan menentukan cluster dengan jarak paling kecil. Batasan masalahnya adalah input harus berupa koordinat tiga dimensi yang valid, dan program harus menggunakan fungsi agar perhitungan jarak serta penentuan cluster lebih terstruktur.

6. Program 6 – Interval Konfidensi

Program ini membahas perhitungan interval konfidensi untuk proporsi populasi berdasarkan proporsi sampel dan tingkat signifikansi yang diberikan. Program menggunakan rumus interval konfidensi dengan nilai z yang sudah ditentukan untuk tingkat kepercayaan 10 persen dan 5 persen, lalu menampilkan batas bawah dan batas atas interval. Batasan masalahnya adalah nilai proporsi sampel harus berada pada rentang 0 sampai 1, dan jika tidak memenuhi syarat tersebut maka program harus menampilkan pesan kesalahan.

2.2. Tabel Input-Proses-Output

Program	Input	Proses	Output
1	Teks sembarang.	Memisahkan teks berdasarkan spasi (kata) dan titik (kalimat).	Jumlah kata dan kalimat.
2	Tidak ada (<i>hardcoded</i>).	Menyimpan nilai teks pada soal ke dalam variabel K1, K2, K3, dan K4	Cetakan 4 kalimat yang sesuai dengan soal.
3	Koefisien a, b, c.	Menghitung diskriminan $D = b^2 - 4ac$ dan mencari akar menggunakan rumus kuadrat.	Akar real (3 desimal) atau imajiner.
4	NIP (18 digit).	Memotong teks (<i>Slicing string</i>) untuk mengambil data tahun, bulan, dan tanggal.	Tanggal, bulan, tahun lahir.
5	Titik U(x_1, x_2, x_3).	Menghitung jarak Euclidean ke tiga pusat <i>cluster</i> $A(2,1,3), B(1, -4,6),$ dan $C(-2,3, -2)$	Nama <i>cluster</i> terdekat atau status di antara <i>cluster</i> .

6	$\hat{p}, n, \text{ dan } \alpha$	Menghitung margin error dengan $\hat{p} - z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$ $< p < \hat{p} + z_{\alpha/2} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$	Batas bawah dan batas atas.
---	-----------------------------------	---	-----------------------------

2.3. Pembagian Tugas Kelompok

1. **Muhammad Nizar Aflah:** Membuat logika R dan Python program (1,3,5), membuat flowchart, dan menyusun laporan.
2. **Danish Prasetya:** Membuat repository github, membuat flowchart, membuat PPT dan menyusun laporan .
3. **Erga Selviana Dewi:** Membuat pseudocode, logika python program(2,3,4), dan menyusun laporan.
4. **Aurella Kristy Sitepu:** Membuat Pseudocode, logika R program (2,3,4), dan menyusun laporan.

BAB III

PSEUDOCODE DAN FLOWCHART

3.1. Program 1

1. Pseudocode

START

FUNCTION hitung_kata_dan_kalimat(teks)

Pisahkan teks berdasarkan spasi
Simpan hasil pemisahan ke daftar_kata

Hitung jumlah elemen pada daftar_kata
Simpan sebagai jumlah_kata

Pisahkan teks berdasarkan tanda titik (.)
Simpan hasil pemisahan ke daftar_kalimat

jumlah_kalimat $\leftarrow$ 0

FOR setiap kalimat dalam daftar_kalimat DO

IF kalimat tidak kosong THEN

jumlah_kalimat $\leftarrow$ jumlah_kalimat + 1

ENDIF

ENDFOR

Tampilkan:

"Teks tersebut memuat"

jumlah_kalimat

"kalimat dan"

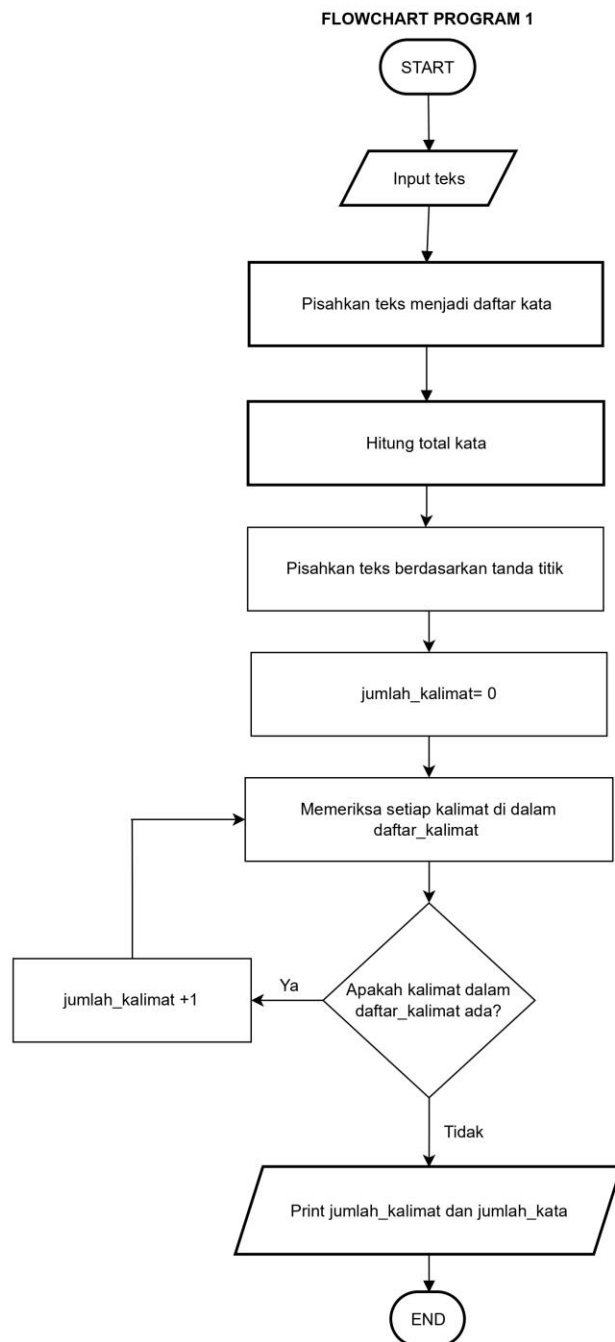
jumlah_kata

"kata"

END FUNCTION

END

2, Flowchart



3.2. Program 2

1. Pseudocode

START

K1 ← "Saya tak 'kan menyerah."

K2 ← 'Ia berkata, "Aku menyayangimu."'

K3 ← "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""

K4 ← "Surat keputusan itu bernomor 62/UN.34/19/2023."

Tampilkan K1

Tampilkan K2

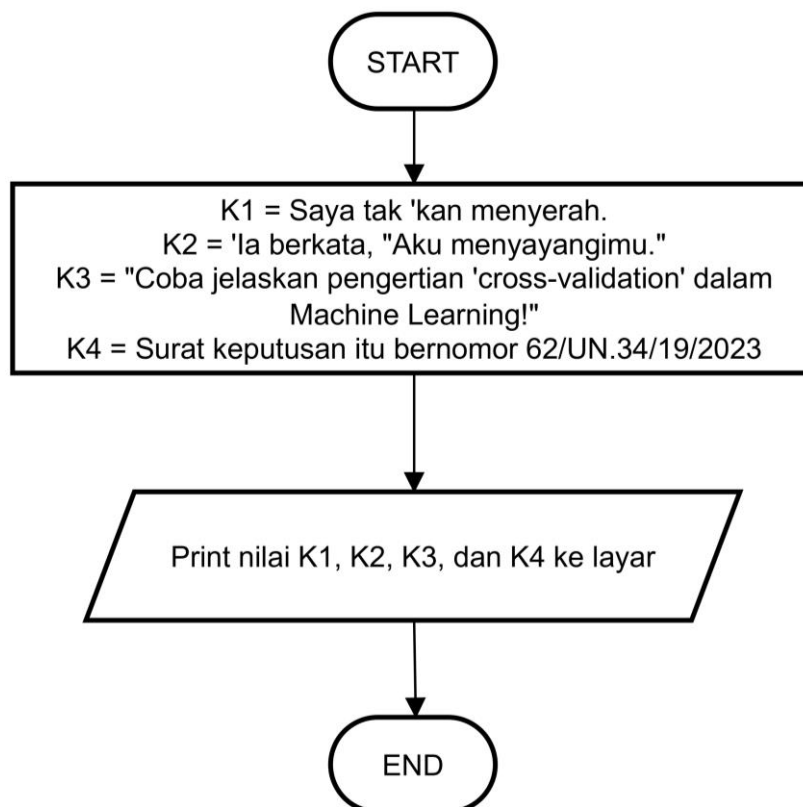
Tampilkan K3

Tampilkan K4

END

2. Flowchart

FLOWCHART PROGRAM 2



3.3. Program 3

1. Pseudocode

START

FUNCTION hitung_akar_kuadrat(koefisien_a, koefisien_b, koefisien_c)

 diskriminan ← $(koefisien_b^2) - (4 \times koefisien_a \times koefisien_c)$

```

IF diskriminan > 0 THEN

    akar_1 ←  $(-\text{koefisien\_b} + \sqrt{\text{diskriminan}}) / (2 \times \text{koefisien\_a})$ 

    akar_2 ←  $(-\text{koefisien\_b} - \sqrt{\text{diskriminan}}) / (2 \times \text{koefisien\_a})$ 

    Tampilkan "Akar real"

    Tampilkan akar_1

    Tampilkan akar_2

ELSE IF diskriminan = 0 THEN

    akar ←  $-\text{koefisien\_b} / (2 \times \text{koefisien\_a})$ 

    Tampilkan "Akar real kembar"

    Tampilkan akar

ELSE

    Tampilkan "Persamaan memiliki akar-akar imajiner"

ENDIF

END FUNCTION

Panggil fungsi hitung_akar_kuadrat(1, -4, 4)

Panggil fungsi hitung_akar_kuadrat(2, -13, 15)

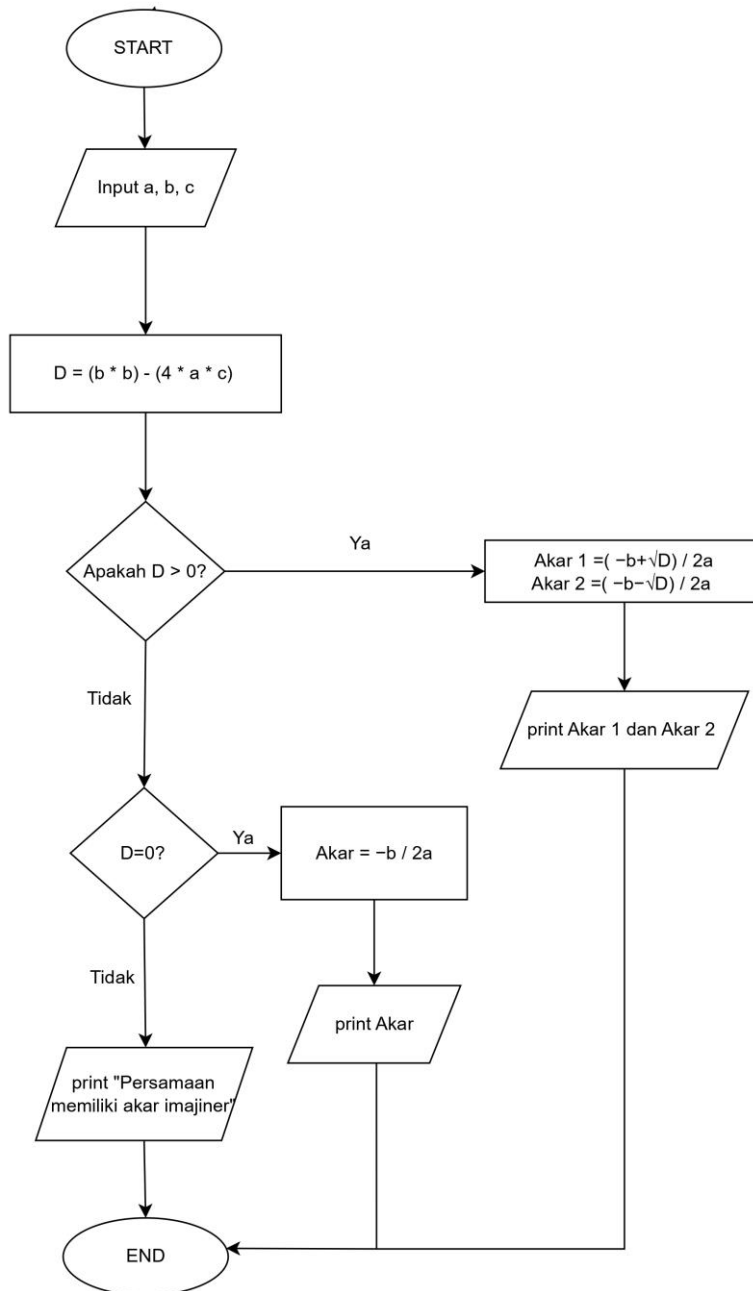
Panggil fungsi hitung_akar_kuadrat(3, -10, 25)

END

```

2. Flowchart

FLOWCHART PROGRAM 3



3.4. Program 4

1. Pseudocode

ALGORITMA Analisis_Tanggal_Lahir_NIP

DEKLARASI:

nip : STRING {Input dari user}

```
tahun, bulan : INTEGER
tanggal      : INTEGER
nama_bulan   : STRING
daftar_bulan : ARRAY OF STRING [1..12]
```

DESKRIPSI:

```
{ Inisialisasi daftar nama bulan }
daftar_bulan ← ["Januari", "Februari", "Maret", "April", "Mei", "Juni",
                "Juli", "Agustus", "September", "Oktober", "November", "Desember"]
```

```
{ Membaca input data NIP }
```

```
BACA nip
```

```
{ Ekstraksi komponen tanggal, bulan, dan tahun lahir menggunakan substring }
```

```
tahun ← KONVERSI_KE_ANGKA(SUBSTRING(nip, 1, 4))
```

```
bulan ← KONVERSI_KE_ANGKA(SUBSTRING(nip, 5, 6))
```

```
tanggal ← KONVERSI_KE_ANGKA(SUBSTRING(nip, 7, 8))
```

```
{ Validasi apakah nomor bulan masuk akal }
```

```
IF (bulan >= 1) AND (bulan <= 12) THEN
```

```
    nama_bulan ← daftar_bulan[bulan]
```

```
    CETAK "Tanggal lahir ASN: ", tanggal, nama_bulan, tahun
```

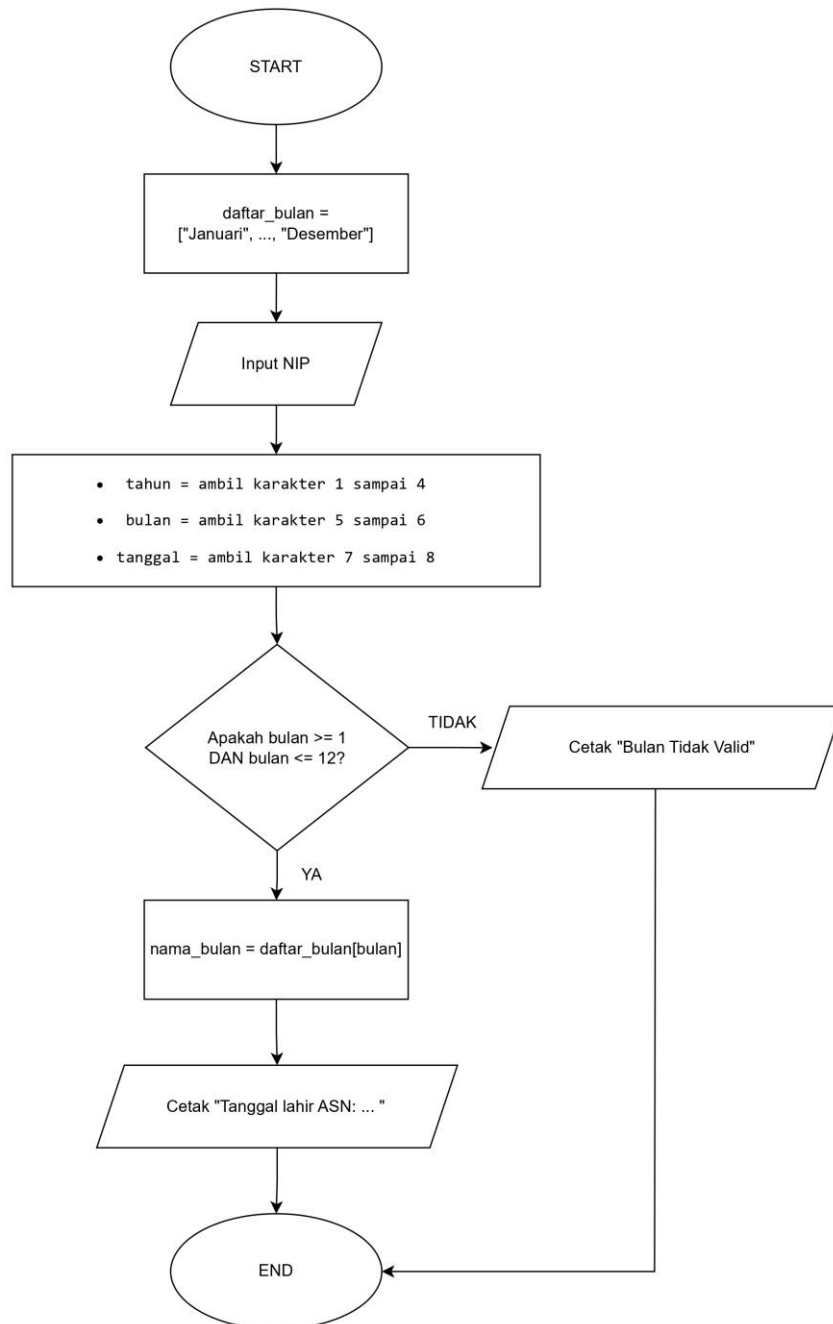
```
ELSE
```

```
    CETAK "Bulan Tidak Valid"
```

```
ENDIF
```

2. Flowchart

FLOWCHART PROGRAM 4



3.5. Program 5

1. Pseudocode

START

ALGORITMA Klasifikasi_Cluster_Titik

DEKLARASI:

x1, x2, x3 : REAL {Input koordinat titik U dari user}

A, B, C, U : ARRAY OF REAL [1..3]

dA, dB, dC : REAL

jarak_min : REAL

cluster_terpilih : LIST OF STRING

DESKRIPSI:

{ Inisialisasi koordinat pusat cluster }

$A \leftarrow [3, 2, 4]$

$B \leftarrow [-2, 5, 1]$

$C \leftarrow [1, -3, -2]$

{ Membaca koordinat titik U }

BACA x1, x2, x3

$U \leftarrow [x1, x2, x3]$

{ Menghitung jarak Euclidean }

$dA \leftarrow \text{AKAR}((x1 - 3)^2 + (x2 - 2)^2 + (x3 - 4)^2)$

$dB \leftarrow \text{AKAR}((x1 - (-2))^2 + (x2 - 5)^2 + (x3 - 1)^2)$

$dC \leftarrow \text{AKAR}((x1 - 1)^2 + (x2 - (-3))^2 + (x3 - (-2))^2)$

{ Cetak nilai jarak }

CETAK "Jarak ke A: ", dA

CETAK "Jarak ke B: ", dB

CETAK "Jarak ke C: ", dC


```

{ Mencari jarak terkecil }
jarak_min ← MINIMUM(dA, dB, dC)

{ Inisialisasi list kosong untuk menampung nama cluster }
cluster_terpilih ← []

{ Pengecekan cluster terdekat }
IF dA == jarak_min THEN
    TAMBAHKAN "Cluster A" KE DALAM cluster_terpilih
ENDIF

IF dB == jarak_min THEN
    TAMBAHKAN "Cluster B" KE DALAM cluster_terpilih
ENDIF

IF dC == jarak_min THEN
    TAMBAHKAN "Cluster C" KE DALAM cluster_terpilih
ENDIF

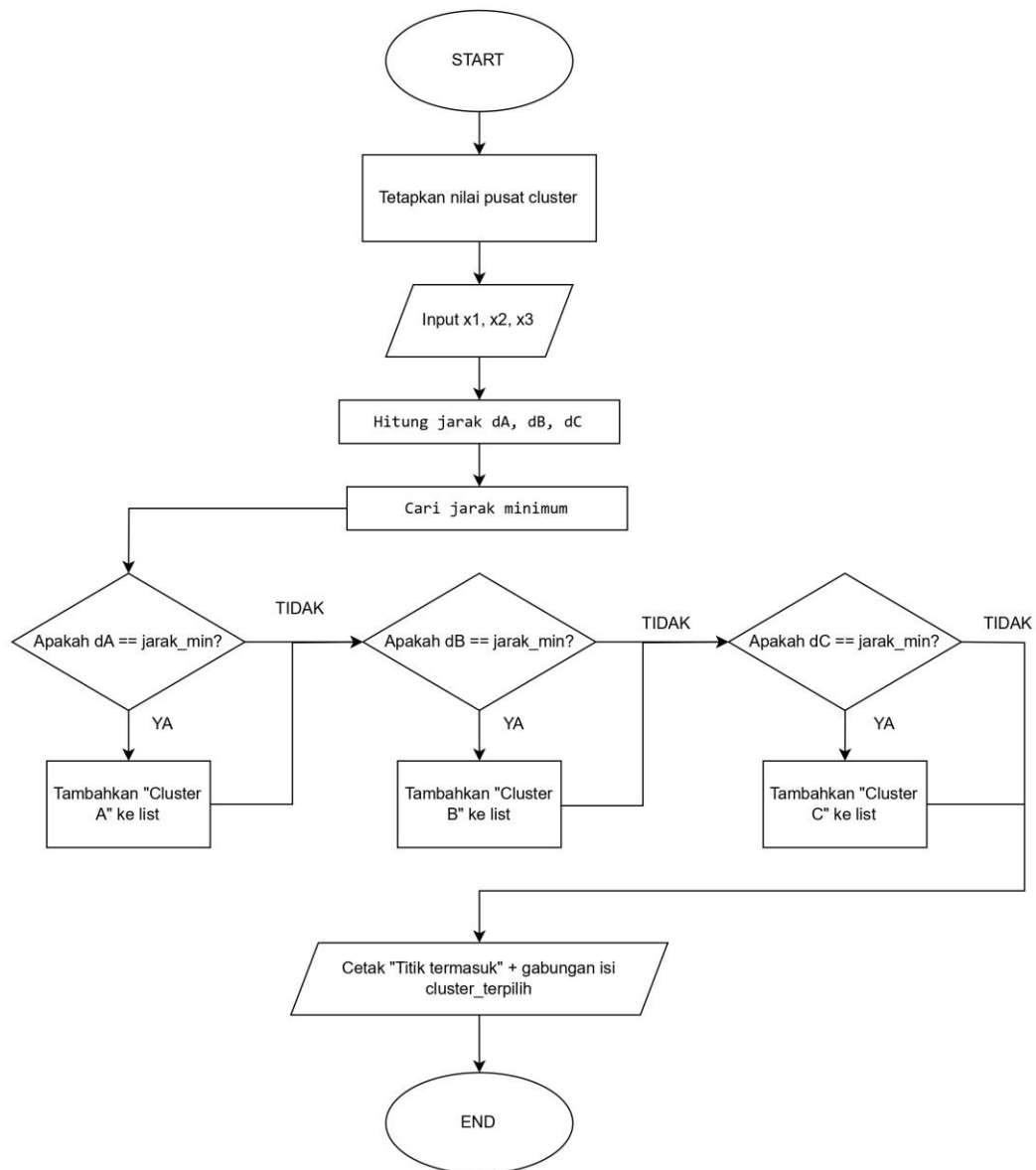
{ Cetak hasil akhir }
CETAK "Titik termasuk ", GABUNGGAN(cluster_terpilih, " dan ")

END

```

2. Flowchart

FLOWCHART PROGRAM 5



3.6. Program 6

1. Pseudocode

START

FUNCTION interval_konfidensi(p_hat, n, alpha)

IF p_hat < 0 OR p_hat > 1 THEN

```

    Tampilkan "Error: Proporsi harus berada pada rentang 0 sampai 1"
    STOP FUNCTION
ENDIF

IF alpha = 0.05 THEN
     $z \leftarrow 1.96$ 

ELSE IF alpha = 0.10 THEN
     $z \leftarrow 1.645$ 

ELSE
    Tampilkan "Error: Alpha harus bernilai 0.05 atau 0.10"
    STOP FUNCTION
ENDIF

margin_error  $\leftarrow z \times \sqrt{(p\_hat \times (1 - p\_hat)) / n}$ 

batas_bawah  $\leftarrow p\_hat - margin\_error$ 

batas_atas  $\leftarrow p\_hat + margin\_error$ 

Tampilkan "Proporsi Sampel =", p_hat

Tampilkan "Ukuran Sampel =", n

Tampilkan "Alpha =", alpha

Tampilkan "Interval Konfidensi = (", batas_bawah, ",", batas_atas, ")"

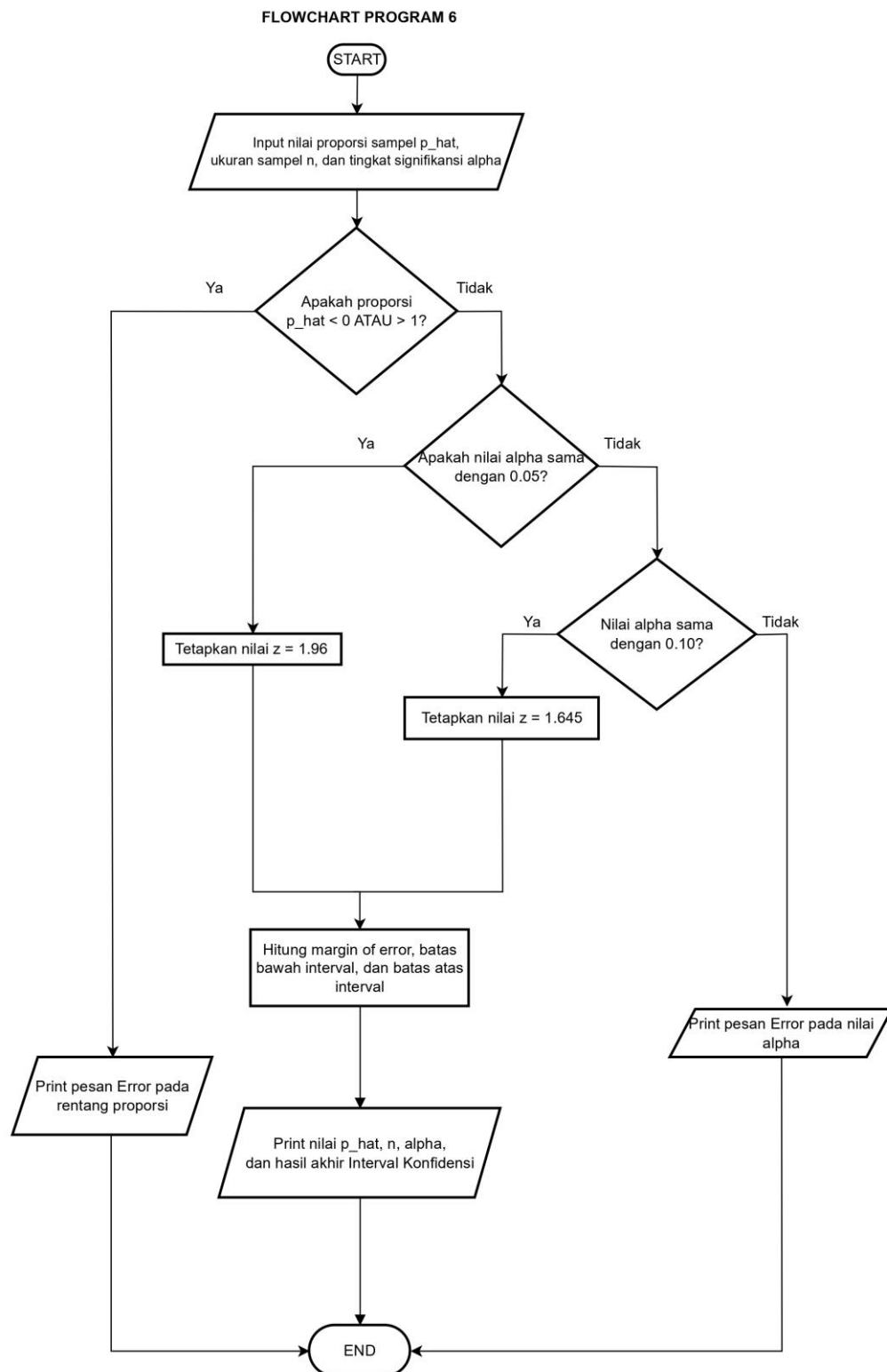
END FUNCTION

Panggil fungsi interval_konfidensi(0.60, 100, 0.05)

END

```

2. Flowchart



BAB IV

IMPLEMENTASI PROGRAM

4.1. Program 1

1. Python

```
def hitung_kata_dan_kalimat(teks):
    daftar_kata = teks.split()
    jumlah_kata = len(daftar_kata)

    daftar_kalimat = teks.split(".")
    jumlah_kalimat = 0

    for kalimat in daftar_kalimat:
        if kalimat.strip() != "":
            jumlah_kalimat += 1

    print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan
          {jumlah_kata} kata.")

# Pemanggilan fungsi
hitung_kata_dan_kalimat("Ketuhanan Yang Maha Esa. Kemanusiaan yang adil dan beradab. Persatuan Indonesia.")
hitung_kata_dan_kalimat("Fungsi adalah blok kode yang diberi nama dan digunakan untuk menjalankan tugas tertentu. Fungsi dapat menerima input berupa parameter atau argumen, memproses input tersebut, kemudian menghasilkan output.")
hitung_kata_dan_kalimat("Aku berlari menjauh dari hal yang tak lagi utuh. Tetes air mata serupa hujan basuhku sepanjang jalan. Dalam leleh kuberharap agar Tuhan berikan aku lupa. Namun itu tak didapatkan kisah lain yang Ia berikan.")

"""
Teks tersebut memuat 3 kalimat dan 11 kata.
Teks tersebut memuat 2 kalimat dan 27 kata.
Teks tersebut memuat 4 kalimat dan 34 kata.
"""
```

2. R

```
hitung_kata_dan_kalimat <- function(teks) {

    daftar_kata <- strsplit(teks, "\\s+")[1]
    jumlah_kata <- length(daftar_kata)

    daftar_kalimat <- strsplit(teks, "\\.[1]")
    jumlah_kalimat <- 0

    for (kalimat in daftar_kalimat) {
        if (trimws(kalimat) != "") {
            jumlah_kalimat <- jumlah_kalimat + 1
        }
    }

    cat("Teks tersebut memuat", jumlah_kalimat, "kalimat dan", jumlah_kata, "kata.\n")
}
```

```

> # Contoh pemanggilan fungsi
> hitung_kata_dan_kalimat("Ketuhanan Yang Maha Esa. Kemanusiaan yang adil dan beradab. Persatuan Indone:
ia.")
Teks tersebut memuat 3 kalimat dan 11 kata.
> hitung_kata_dan_kalimat("Fungsi adalah blok kode yang diberi nama dan digunakan untuk menjalankan tug
s tertentu. Fungsi dapat menerima input berupa parameter atau argumen, memproses input tersebut, kemudi
n menghasilkan output.")
Teks tersebut memuat 2 kalimat dan 27 kata.
> hitung_kata_dan_kalimat("aku berlari menjauh dari hal yang tak lagi utuh. tetes air mata serupa hujan
basahiku sepanjang jalan. dalam lelah kuberharap agar tuhan berikan aku lupa. namun itu tak kudapatkan l
isah lain yang ia berikan.")
Teks tersebut memuat 4 kalimat dan 34 kata.
>

```

4.2. Program 2

1. Python

```

K1="Saya tak 'kan menyerah."
K2='Ia berkata, "Aku menyayangimu."'
K3="\Coba jelaskan pengertian 'cross-validation' dalam Machine
Learning!\n"
K4="Surat keputusan itu bernomor 62/UN.34/19/2023."

```

```

print(K1)
print(K2)
print(K3)
print(K4)

```

```

print(K1)
print(K2)
print(K3)
print(K4)

```

```

Saya tak 'kan menyerah.
Ia berkata, "Aku menyayangimu."
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
Surat keputusan itu bernomor 62/UN.34/19/2023.

```

2. R

```

K1 <- "Saya tak 'kan menyerah."
K2 <- 'Ia berkata, "Aku menyayangimu."'
K3 <- "\Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\n"
K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."

```

```

> cat(K1, "\n")
Saya tak 'kan menyerah.
> cat(K2, "\n")
Ia berkata, "Aku menyayangimu."
> cat(K3, "\n")
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!
> cat(K4, "\n")
Surat keputusan itu bernomor 62/UN.34/19/2023.
>

```

4.3. Program 3

1. Python

```
import math

def hitung_akar_kuadrat(koefisien_a, koefisien_b, koefisien_c):

    diskriminan = (koefisien_b**2) - (4 * koefisien_a * koefisien_c)

    if diskriminan > 0:

        akar_1 = (-koefisien_b + math.sqrt(diskriminan)) / (2 * koefisien_a)
        akar_2 = (-koefisien_b - math.sqrt(diskriminan)) / (2 * koefisien_a)

        print("Persamaan tersebut memiliki akar real:")
        print(f"Akar 1 = {akar_1:.3f}")
        print(f"Akar 2 = {akar_2:.3f}\n")

    elif diskriminan == 0:

        akar = -koefisien_b / (2 * koefisien_a)

        print("Persamaan tersebut memiliki akar real kembar:")
        print(f"Akar = {akar:.3f}\n")

    else:

        print("Persamaan tersebut hanya memiliki akar-akar imajiner\n")

#Pemanggilan fungsi
hitung_akar_kuadrat(1, -4, 4)
hitung_akar_kuadrat(2, -13, 15)
hitung_akar_kuadrat(3, -10, 25)
```

-
- Persamaan tersebut memiliki akar real kembar:
Akar = 2.000

Persamaan tersebut memiliki akar real:
Akar 1 = 5.000
Akar 2 = 1.500

Persamaan tersebut hanya memiliki akar-akar imajiner

2. R

```
hitung_akar_kuadrat <- function(koefisien_a, koefisien_b, koefisien_c) {  
  diskriminan <- (koefisien_b ^ 2) - (4 * koefisien_a * koefisien_c)  
  
  if (diskriminan > 0) {  
    akar_1 <- (-koefisien_b + sqrt(diskriminan)) / (2 * koefisien_a)  
    akar_2 <- (-koefisien_b - sqrt(diskriminan)) / (2 * koefisien_a)  
  
    cat("Persamaan tersebut memiliki akar real:\n")  
    cat(sprintf("Akar 1 = %.3f\n", akar_1))  
    cat(sprintf("Akar 2 = %.3f\n", akar_2))  
  
  } else if (diskriminan == 0) {  
  
    akar <- -koefisien_b / (2 * koefisien_a)  
  
    cat("Persamaan tersebut memiliki akar real kembar:\n")  
    cat(sprintf("Akar = %.3f\n", akar))  
  
  } else {  
  
    cat("Persamaan tersebut hanya memiliki akar-akar imajiner\n")  
  
  }  
}
```

```
' J  
>  
> hitung_akar_kuadrat(1,-4,4)  
Persamaan tersebut memiliki akar real kembar:  
Akar = 2.000  
> hitung_akar_kuadrat(2,-13,15)  
Persamaan tersebut memiliki akar real:  
Akar 1 = 5.000  
Akar 2 = 1.500  
> hitung_akar_kuadrat(3,-10,25)  
Persamaan tersebut hanya memiliki akar-akar imajiner
```


4.4. Program 4

1. Python

```
def tanggal_lahir(nip):
    nip_str = str(nip)

    tahun = int(nip_str[0:4])
    bulan = int(nip_str[4:6])
    tanggal = int(nip_str[6:8])

    daftar_bulan = [
        "Januari",
        "Februari",
        "Maret",
        "April",
        "Mei",
        "Juni",
        "Juli",
        "Agustus",
        "September",
        "Oktober",
        "November",
        "Desember",
    ]

    if 1 <= bulan <= 12:
        nama_bulan = daftar_bulan[bulan - 1]
        print(f"Tanggal lahir ASN: {tanggal} {nama_bulan} {tahun}\n")
    else:
        print("Bulan Tidak Valid\n")

#Pemanggilan fungsi
tanggal_lahir(199210252015032015)
tanggal_lahir(198540122010011003)
tanggal_lahir(199812012023021001)
```

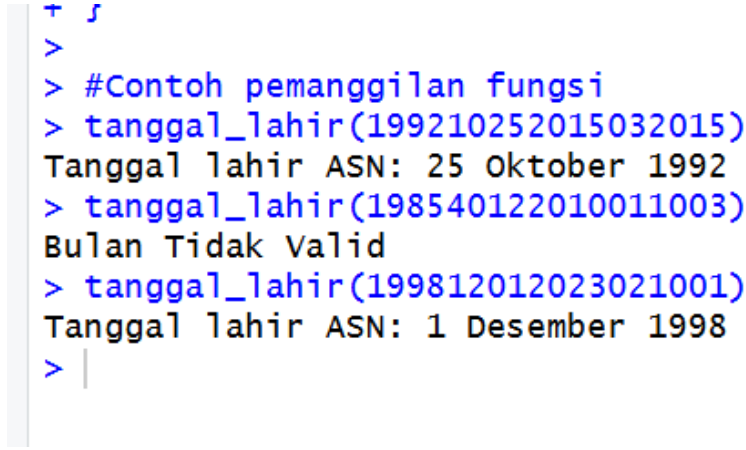
• Tanggal lahir ASN: 25 Oktober 1992

Bulan Tidak Valid

Tanggal lahir ASN: 1 Desember 1998

2. R

```
tanggal_lahir <- function(nip) {  
  
  nip_str <- as.numeric(nip)  
  
  tahun <- as.numeric(substr(nip_str, 1, 4))  
  bulan <- as.numeric(substr(nip_str, 5, 6))  
  tanggal <- as.numeric(substr(nip_str, 7, 8))  
  
  daftar_bulan <- c("Januari", "Februari", "Maret", "April", "Mei", "Juni",  
                    "Juli", "Agustus", "September", "Oktober", "November", "Desember")  
  
  if (bulan >= 1 && bulan <= 12) {  
    nama_bulan <- daftar_bulan[bulan]  
    cat("Tanggal lahir ASN:", tanggal, nama_bulan, tahun, "\n")  
  } else {  
    cat("Bulan Tidak Valid\n")  
  }  
}
```



```
+ ↵  
>  
> #Contoh pemanggilan fungsi  
> tanggal_lahir(199210252015032015)  
Tanggal lahir ASN: 25 Oktober 1992  
> tanggal_lahir(198540122010011003)  
Bulan Tidak Valid  
> tanggal_lahir(199812012023021001)  
Tanggal lahir ASN: 1 Desember 1998  
> |
```

4.5. Program 5

1. Python

```
import numpy as np  
  
def cluster_titik(x1, x2, x3):  
    A = np.array([3, 2, 4])  
    B = np.array([-2, 5, 1])
```

```

C = np.array([1, -3, -2])

U = np.array([x1, x2, x3])

dA = np.sqrt(np.sum((U - A) ** 2))
dB = np.sqrt(np.sum((U - B) ** 2))
dC = np.sqrt(np.sum((U - C) ** 2))

print(f"Jarak ke A: {dA:.4f}")
print(f"Jarak ke B: {dB:.4f}")
print(f"Jarak ke C: {dC:.4f}")

jarak_min = min(dA, dB, dC)
cluster_terpilih = []

if dA == jarak_min:
    cluster_terpilih.append("Cluster A")
if dB == jarak_min:
    cluster_terpilih.append("Cluster B")
if dC == jarak_min:
    cluster_terpilih.append("Cluster C")

output_cluster = " dan ".join(cluster_terpilih)
print(f"Titik termasuk {output_cluster}\n")

#Pemanggilan fungsi
cluster_titik(4, 3, 7)
cluster_titik(3, -7, -1)
cluster_titik(0.5, 3.5, 2.5)

```

```

... Jarak ke A: 3.3166
    Jarak ke B: 8.7178
    Jarak ke C: 11.2250
    Titik termasuk Cluster A

    Jarak ke A: 10.2956
    Jarak ke B: 13.1529
    Jarak ke C: 4.5826
    Titik termasuk Cluster C

    Jarak ke A: 3.2787
    Jarak ke B: 3.2787
    Jarak ke C: 7.9215
    Titik termasuk Cluster A dan Cluster B

```

2. R

```

cluster_titik <- function(x1, x2, x3) {
  A <- c(3, 2, 4)

```

```

B <- c(-2, 5, 1)
C <- c(1, -3, -2)

U <- c(x1, x2, x3)

dA <- sqrt(sum((U - A)^2))
dB <- sqrt(sum((U - B)^2))
dC <- sqrt(sum((U - C)^2))

print(paste("Jarak ke A:", round(dA, 4)))
print(paste("Jarak ke B:", round(dB, 4)))
print(paste("Jarak ke C:", round(dC, 4)))

jarak_min <- min(dA, dB, dC)
cluster_terpilih <- c()

if (dA == jarak_min) {
  cluster_terpilih <- c(cluster_terpilih, "Cluster A")
}
if (dB == jarak_min) {
  cluster_terpilih <- c(cluster_terpilih, "Cluster B")
}
if (dC == jarak_min) {
  cluster_terpilih <- c(cluster_terpilih, "Cluster C")
}

cat("Titik termasuk", paste(cluster_terpilih, collapse = " dan "), "\n\n")
}

```

```

> cluster_titik(4, 3, 7)
[1] "Jarak ke A: 3.3166"
[1] "Jarak ke B: 8.7178"
[1] "Jarak ke C: 11.225"
Titik termasuk Cluster A

> cluster_titik(3,-7,-1)
[1] "Jarak ke A: 10.2956"
[1] "Jarak ke B: 13.1529"
[1] "Jarak ke C: 4.5826"
Titik termasuk Cluster C

> cluster_titik(0.5, 3.5, 2.5)
[1] "Jarak ke A: 3.2787"
[1] "Jarak ke B: 3.2787"
[1] "Jarak ke C: 7.9215"
Titik termasuk Cluster A dan Cluster B

> |

```

4.6. Program 6

1. Python

```

import math

def interval_konfidensi(p_hat, n, alpha):

    if p_hat < 0 or p_hat > 1:
        print("Error: Proporsi harus berada pada rentang 0 sampai 1.\n")
        return

    if alpha == 0.05:
        z = 1.96
    elif alpha == 0.10:
        z = 1.645
    else:
        print("Error: Alpha harus bernilai 0.05 atau 0.10.\n")
        return

    margin_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)

    batas_bawah = p_hat - margin_error
    batas_atas = p_hat + margin_error

    print(f"Proporsi Sampel = {p_hat}")
    print(f"Ukuran Sampel = {n}")
    print(f"Alpha = {alpha}")
    print(f"Interval Konfidensi = ( {batas_bawah:.3f} , {batas_atas:.3f}
)\n")

```

```
#Pemanggilan fungsi
interval_konfidensi(0.4, 100, 0.05)
interval_konfidensi(0.6, 250, 0.01)
interval_konfidensi(1.2, 150, 0.10)
```

```
... Proporsi Sampel = 0.4
    Ukuran Sampel = 100
    Alpha = 0.05
    Interval Konfidensi = ( 0.304 , 0.496 )

    Error: Alpha harus bernilai 0.05 atau 0.10.

    Error: Proporsi harus berada pada rentang 0 sampai 1.
```

2. R

```
interval_konfidensi <- function(p_hat, n, alpha) {
  if (p_hat < 0 || p_hat > 1) {
    cat("Error: Proporsi harus berada pada rentang 0 sampai 1.\n")
    return()
  }
  if (alpha == 0.05) {
    z <- 1.96
  } else if (alpha == 0.10) {
    z <- 1.645
  } else {
    cat("Error: Alpha harus bernilai 0.05 atau 0.10.\n")
    return()
  }

  margin_error <- z * sqrt((p_hat * (1 - p_hat)) / n)
  batas_bawah <- p_hat - margin_error
  batas_atas <- p_hat + margin_error

  cat("Proporsi Sampel =", p_hat, "\n")
  cat("Ukuran Sampel =", n, "\n")
  cat("Alpha =", alpha, "\n")
}
```

```

cat("Interval Konfidensi = (",round(batas_bawah, 3),",",round(batas_atas, 3),")\n")
}

> # Contoh data
> interval_konfidensi(0.4, 100, 0.05)
Proporsi Sampel = 0.4
Ukuran Sampel = 100
Alpha = 0.05
Interval Konfidensi = ( 0.304 , 0.496 )
> interval_konfidensi(0.6, 250, 0.01)
Error: Alpha harus bernilai 0.05 atau 0.10.
NULL
> interval_konfidensi(1.2, 150, 0.10)
Error: Proporsi harus berada pada rentang 0 sampai 1.
NULL
> |

```

BAB V

PENJELASAN LINE-BY-LINE-CODE

5.1. Program 1

No	Baris Kode Python	Penjelasan
1	<code>def hitung_kata_dan_kalimat(teks):</code>	Mendefinisikan fungsi bernama <code>hitung_kata_dan_kalimat</code> yang menerima satu argumen berupa teks.
2	<code>daftar_kata = teks.split()</code>	Memisahkan teks berdasarkan spasi dan menyimpannya dalam bentuk list <code>daftar_kata</code> .
3	<code>jumlah_kata = len(daftar_kata)</code>	Menghitung jumlah elemen dalam list <code>daftar_kata</code> untuk mendapatkan total kata
4	<code>daftar_kalimat = teks.split(".")</code>	Memisahkan teks berdasarkan tanda titik (.) untuk memecahnya menjadi kalimat-kalimat.
5	<code>jumlah_kalimat = 0</code>	Menginisialisasi variabel <code>jumlah_kalimat</code> dengan nilai 0 sebelum iterasi.
6	<code>for kalimat in daftar_kalimat:</code>	Memulai perulangan untuk memeriksa setiap kalimat di dalam list <code>daftar_kalimat</code> .
7	<code>if kalimat.strip() != "":</code>	Memeriksa apakah kalimat tersebut tidak kosong setelah spasi di awal/akhir dihapus (<code>strip</code>).
8	<code>jumlah_kalimat += 1</code>	Menambah nilai <code>jumlah_kalimat</code> sebanyak 1 jika kondisi kalimat tidak kosong terpenuhi.
9	<code>print(f'Teks tersebut memuat...</code>	Menampilkan hasil akhir <code>jumlah_kalimat</code> dan <code>jumlah_kata</code> ke layar.

No	Baris Kode R	Penjelasan
1	<code>hitung_kata_dan_kalimat <- function(teks) {</code>	Membuat fungsi <code>hitung_kata_dan_kalimat</code> dengan parameter masukan teks.

2	<code>daftar_kata <- strsplit(teks, "\\s+") [[1]]</code>	Memecah teks menjadi array kata menggunakan regular expression (spasi/whitespace).
3	<code>jumlah_kata <- length(daftar_kata)</code>	Menghitung panjang array kata untuk mengetahui total jumlah kata.
4	<code>daftar_kalimat <- strsplit(teks, "\\.")[[1]]</code>	Memecah teks berdasarkan karakter titik (.) untuk mendata kalimat.
5	<code>jumlah_kalimat <- 0</code>	Menginisialisasi nilai awal perhitungan jumlah kalimat menjadi 0
6	<code>for (kalimat in daftar_kalimat) {</code>	Melakukan iterasi atau perulangan pada setiap elemen dalam array kalimat.
7	<code>if (trimws(kalimat) != "") {</code>	Memeriksa agar kalimat yang hanya berisi spasi kosong tidak dihitung menggunakan trimws.
8	<code>jumlah_kalimat <- jumlah_kalimat + 1</code>	Menambahkan angka 1 pada jumlah_kalimat jika kalimat tersebut terdeteksi valid/tidak kosong.
9	<code>cat("Teks tersebut memuat"...</code>	Mencetak output jumlah kalimat dan jumlah kata ke konsol.

5.2. Program 2

No	Baris Kode Python	Penjelasan
1	<code>K1="Saya tak 'kan menyerah."</code>	Menyimpan sebuah string berisi kutipan dengan tanda petik tunggal di dalamnya ke dalam variabel
2	<code>K2='Ia berkata, "Aku menyayangimu."'</code>	Menyimpan string dengan tanda petik ganda di dalamnya ke dalam variabel K2 menggunakan kutip tunggal di luar.
3	<code>K3="\Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\\"</code>	Menyimpan string ke K3 dengan menggunakan escape character \ agar tanda kutip ganda bisa dicetak.
4	<code>K4="Surat keputusan itu bernomor 62/UN.34/19/2023."</code>	Menyimpan teks string biasa berupa nomor surat ke dalam variabel K4.

5	<code>print(K1) hingga print(K4)</code>	Mencetak isi dari masing-masing variabel K1, K2, K3, dan K4 ke layar
---	---	--

No	Baris Kode R	Penjelasan
1	<code>K1<-"Saya tak 'kan menyerah."</code>	Mendeklarasikan variabel K1 yang diisi dengan string berisikan petik tunggal.
2	<code>K2<-'Ia berkata, "Aku menyayangimu."'"</code>	Mendeklarasikan variabel K2 yang diisi dengan teks kutipan langsung.
3	<code>K3<-\"Coba jelaskan pengertian...\"</code>	Mendeklarasikan K3 yang berisikan teks dengan escape sequence untuk kutip ganda.
4	<code>K4<-"Surat keputusan itu bernomor..."</code>	Menyimpan format nomor surat ke dalam variabel string K4.
5	<code>cat(K1) hingga cat(K4)</code>	Perintah untuk mencetak data string dari K1 sampai K4 pada konsol.

5.3. Program 3

No	Baris Kode Python	Penjelasan
1	<code>def hitung_akar_kuadrat(koefisien_a...):</code>	Mendefinisikan fungsi kalkulasi akar persamaan kuadrat dengan 3 parameter (a, b, c).
2	<code>diskriminan = (koefisien_b ** 2) - ...</code>	Menghitung nilai diskriminan ($D=b^2-4ac$).
3	<code>if diskriminan > 0:</code>	Mengevaluasi apakah persamaan memiliki dua akar real yang berbeda ($D > 0$).
4	<code>akar_1 = (-koefisien_b + math.sqrt...</code>	Menghitung akar pertama menggunakan rumus ABC dengan menjumlahkan akar diskriminan.
5	<code>akar_2 = (-koefisien_b - math.sqrt...</code>	Menghitung akar kedua menggunakan rumus ABC dengan mengurangi akar diskriminan

6	<code>elif diskriminan == 0:</code>	Mengevaluasi kemungkinan persamaan memiliki akar real kembar ($D = 0$).
7	<code>akar = -koefisien_b / (2 * koefisien_a)</code>	Menghitung nilai akar kembar tersebut dengan rumus $-b/2a$
8	<code>else:</code>	Kondisi terakhir jika nilai diskriminan negatif ($D < 0$).
9	<code>print("Persamaan memiliki akar imajiner")</code>	Mencetak peringatan bahwa akar persamaan bersifat imajiner/kompleks.

No	Baris Kode R	Penjelasan
1	<code>hitung_akar_kuadrat <- function(...) {</code>	Mendefinisikan fungsi perhitungan akar kuadrat di R dengan input 3 parameter koefisien.
2	<code>diskriminan <- (koefisien_b^2) - ...</code>	Kalkulasi nilai diskriminan berdasarkan input dari pengguna.
3	<code>if (diskriminan > 0) {</code>	Logika percabangan untuk mengecek jika terdapat dua akar real ($D > 0$).
4	<code>akar_1 <- (-koefisien_b + sqrt...</code>	Menghitung akar pertama persamaan.
5	<code>akar_2 <- (-koefisien_b - sqrt...</code>	Menghitung akar kedua persamaan.
6	<code>} else if (diskriminan == 0) {</code>	Logika percabangan alternatif jika diskriminan sama dengan nol.
7	<code>akar <- -koefisien_b / (2*koefisien_a)</code>	Menghitung nilai tunggal jika akar berbentuk kembar.
8	<code>} else {</code>	Logika fallback jika dua kondisi sebelumnya ($D > 0$ dan $D = 0$) gagal, artinya $D < 0$.
9	<code>cat("Persamaan memiliki akar imajiner\n")</code>	Mencetak pesan bahwa akar adalah imajiner karena D kurang dari nol.

5.4. Program 4

No	Baris Kode Python	Penjelasan
1	<code>def tanggal_lahir(nip):</code>	Mendefinisikan fungsi bernama <code>tanggal_lahir</code> yang menerima satu parameter masukan yaitu <code>nip</code> .
2	<code>nip_str = str(nip)</code>	Mengubah tipe data variabel <code>nip</code> (yang awalnya angka/integer) menjadi string agar karakternya dapat dipotong atau diakses berdasarkan indeks.
3	<code>tahun = int(nip_str[0:4])</code>	Mengambil 4 karakter pertama dari string NIP (indeks 0 sampai sebelum 4) dan mengubahnya kembali menjadi integer sebagai data tahun lahir.
4	<code>bulan = int(nip_str[4:6])</code>	Mengambil 2 karakter pertama dari string NIP (indeks 4 sampai sebelum 6) dan mengubahnya kembali menjadi integer sebagai data bulan lahir.
5	<code>tanggal = int(nip_str[6:8])</code>	Mengambil karakter indeks ke-6 dan ke-7 dari string NIP lalu mengubahnya menjadi integer sebagai data tanggal lahir.
6	<code>daftar_bulan = [...]</code>	Membuat sebuah list (array) bernama <code>daftar_bulan</code> yang menampung string nama-nama bulan secara berurutan dari Januari hingga Desember.
7	<code>if 1 <= bulan <= 12:</code>	Melakukan validasi percabangan untuk memeriksa apakah nilai variabel <code>bulan</code> berada dalam rentang yang sah, yaitu antara angka 1 sampai 12.
8	<code>nama_bulan = daftar_bulan[bulan - 1]</code>	Mengambil nama bulan dari <code>daftar_bulan</code> berdasarkan posisi indeksinya. Dikurangi 1 (<code>bulan - 1</code>) karena indeks list pada Python dimulai dari angka 0 (Januari = indeks 0).

9	<code>print(f"Tanggal lahir ASN: {tanggal}...</code>	Menampilkan output teks terformat ke layar yang berisi tanggal, nama bulan, dan tahun lahir jika bulannya valid.
10	<code>else:</code>	Bagian percabangan alternatif yang akan dijalankan jika kondisi nilai bulan di luar rentang 1-12 tidak terpenuhi.
11	<code>print("Bulan Tidak Valid\n")</code>	Menampilkan pesan peringatan ke layar bahwa input bulan yang diekstraksi tidak valid

No	Baris Kode R	Penjelasan
1	<code>tanggal_lahir<- function(nip) {</code>	Membuat sebuah fungsi bernama tanggal_lahir dalam bahasa R dengan parameter input bernama nip.
2	<code>nip_str<- as.character(nip)</code>	Mengonversi data nip menjadi tipe data karakter (string) agar posisi tiap digit angkanya bisa dipotong.
3	<code>tahun<-as.numeric(substr(nip_str, 1, 4))</code>	Memotong string nip_str dari karakter ke-1 hingga ke-4 menggunakan fungsi substr(), lalu mengubah hasilnya menjadi numerik untuk variabel tahun.
4	<code>bulan<-as.numeric(substr(nip_str, 5, 6))</code>	Memotong string dari posisi karakter ke-5 hingga ke-6 dan mengubahnya menjadi nilai numerik untuk variabel bulan.
5	<code>tanggal<- as.numeric(substr(nip_str, 7, 8))</code>	Memotong string dari posisi karakter ke-7 hingga ke-8 dan mengubahnya menjadi nilai numerik untuk variabel tanggal.
6	<code>daftar_bulan<- c("Januari", ...)</code>	Membuat sebuah vektor menggunakan fungsi c() bernama daftar_bulan untuk menyimpan daftar nama bulan dari Januari sampai Desember.
7	<code>if (bulan >= 1 && bulan <= 12) {</code>	Melakukan pengecekan kondisi apakah nilai bulan valid (bernilai minimal 1 dan

		maksimal 12) menggunakan operator logika && (DAN).
8	<code>nama_bulan<- daftar_bulan[bulan]</code>	Mengambil nama bulan dari vektor <code>daftar_bulan</code> berdasarkan nomor bulannya. Tidak perlu dikurangi 1 karena indeks penomoran di bahasa R dimulai dari angka 1 (Januari = indeks 1).
9	<code>cat("Tanggal lahir ASN:", tanggal...</code>	Mencetak gabungan teks hasil ekstraksi (tanggal, nama bulan, tahun) ke konsol/layar.
10	<code>} else {</code>	Menandakan kondisi alternatif apabila hasil pengecekan variabel bulan bernilai salah atau di luar batas 1–12.
11	<code>cat("Blan Tidak Valid\n")</code>	Mencetak pesan peringatan kesalahan ke konsol bahwa bulan yang tertera pada NIP tidak valid.

5.5. Program 5

No	Baris Kode Python	Penjelasan
1	<code>def cluster_titik(x1, x2, x3):</code>	Fungsi yang menampung 3 koordinat sebuah titik dimensi tiga (x1, x2, x3).
2	<code>A = (2, 1, 3); B = ...; C = ...</code>	Mendeklarasikan posisi koordinat acuan untuk titik pusat Cluster A, B, dan C dalam bentuk tuple.
3	<code>U = (x1, x2, x3)</code>	Menyatukan input dari parameter menjadi satu tuple koordinat titik U.
4	<code>dA = math.sqrt(sum((u - a)**2...</code>	Menghitung jarak Euclidean atau jarak titik ke titik dari titik masukan (U) ke pusat Cluster A. (Berlaku serupa untuk dB dan dC)
5	<code>if dA <= dB and dA <= dC:</code>	Memeriksa apakah Jarak ke Cluster A merupakan jarak terkecil/terdekat dibanding cluster lain.

6	<code>print("Titik termasuk Cluster A")</code>	Mencetak konklusi penentuan cluster jika titik paling dekat ke Cluster A. (Sama halnya dengan elif dan else pada cluster B dan C).
---	--	--

No	Baris Kode R	Penjelasan
1	<code>cluster_titik <- function(x1,x2,x3) {</code>	Fungsi untuk mengelompokkan input berdasarkan nilai koordinat X1, X2, dan X3.
2	<code>A <- c(2, 1, 3); B <- ...; C <- ...</code>	Menyimpan posisi pusat cluster (A, B, dan C) menggunakan vector <code>c()</code> .
3	<code>U <- c(x1, x2, x3)</code>	Memasukkan nilai argument parameter dalam satu buah vector pengujian U.
4	<code>dA <- sqrt(sum((U - A)^2))</code>	Perhitungan jarak Euclidean dari titik U ke pusat cluster A di bahasa R. (Berlaku untuk dB dan dC juga).
5	<code>if (dA <= dB && dA <= dC) {</code>	Percabangan pengecekan apakah rentang jarak A lebih pendek ketimbang B maupun C
6	<code>cat("Titik termasuk Cluster A\n")</code>	Output hasil akhir penetapan bahwa titik tersebut ditugaskan kepada Cluster A (Sama untuk kondisi else if di bawahnya)

5.6. Program 6

No	Baris Kode Python	Penjelasan
1	<code>def interval_konfidensi(p_hat, n, alpha):</code>	Definisi fungsi penghitung interval konfidensi berdasarkan proporsi populasi, ukuran sampel, dan tingkat signifikansi alpha.
2	<code>if p_hat < 0 or p_hat > 1:</code>	Logika validasi bahwa parameter probabilitas proporsi (<code>p_hat</code>) harus dalam rentang 0 hingga 1.

3	<code>if alpha == 0.05: z = 1.96</code>	Mengonversi tingkat signifikansi alpha 5% menjadi Z-score/nilai Z kritis sepadan, yakni 1.96.
4	<code>margin_error = z * math.sqrt...</code>	Menghitung nilai Margin of Error menggunakan formula $Z \times \sqrt{\frac{p(1-p)}{n}}$
5	<code>batas_bawah = p_hat - margin_error</code>	Penentuan batas bawah dari interval konfidensi, yaitu proporsi dikurangi Margin Error
6	<code>batas_atas = p_hat + margin_error</code>	Penentuan batas atas dari interval konfidensi, yaitu proporsi ditambah Margin Error
7	<code>print(f"Interval Konfidensi = (...</code>	Mencetak hasil kalkulasi interval dengan membulatkan nilai hingga 3 angka di belakang koma dengan round().

No	Baris Kode R	Penjelasan
1	<code>interval_konfidensi <- function(p_hat, n, alpha) {</code>	Membuat fungsi bernama interval_konfidensi di R dengan tiga argumen masukan (p_hat, n, dan alpha).
2	<code>if (p_hat < 0</code>	Jika
3	<code>if (alpha == 0.05) { z <- 1.96 }</code>	Struktur kontrol untuk menentukan nilai kritis Z. Jika nilai alpha sama dengan 0.05, maka variabel z diisi dengan 1.96.
4	<code>else if (alpha == 0.01) { z <- 2.576 }</code>	Percabangan kedua, jika nilai alpha sama dengan 0.01, maka variabel z diisi dengan nilai 2.576.
5	<code>else { z <- 1.645 }</code>	Kondisi terakhir, jika alpha bernilai selain kedua angka di atas, nilai z disetel default menjadi 1.645.
6	<code>margin_error <- z * sqrt((p_hat * (1 - p_hat)) / n)</code>	Melakukan perhitungan statistik untuk mencari nilai batas kesalahan (Margin of

		Error) dengan fungsi akar kuadrat <code>sqrt()</code> di R.
7	<code>batas_bawah <- p_hat - margin_error</code>	Menentukan titik terendah (rentang minimum) dari estimasi interval konfidensi.
8	<code>batas_atas <- p_hat + margin_error</code>	Menentukan titik tertinggi (rentang maksimum) dari estimasi interval konfidensi.
9	<code>cat("Interval Konfidensi = (", round(batas_bawah, 3), ", ", round(batas_atas, 3), ")\\n")</code>	Mencetak hasil akhir interval ke konsol R dengan format rapi, serta membulatkan angka desimalnya menjadi 3 digit di belakang koma lewat fungsi <code>round()</code> .

BAB VI

PENGUJIAN PROGRAM

6.1. Tabel Pengujian

Program 1

Input	Output Diharapkan	Output Program	Status	Catatan
("Ketuhanan Yang Maha Esa. Kemanusiaan yang adil dan beradab. Persatuan Indonesia.")	3 kalimat dan 11 kata	3 kalimat dan 11 kata	Berhasil	Output sesuai dengan yang diharapkan
("Fungsi adalah blok kode yang diberi nama dan digunakan untuk menjalankan tugas tertentu. Fungsi dapat menerima input berupa parameter atau argumen, memproses input tersebut, kemudian menghasilkan output.")	2 kalimat dan 27 kata	2 kalimat dan 27 kata	Berhasil	
("aku berlari menjauh dari hal yang tak lagi utuh. tetes air mata serupa hujan basahiku sepanjang jalan. dalam lelah kuberharap agar tuhan berikan aku lupa. namun itu tak kudapatkan kisah lain yang ia berikan.")	4 kalimat dan 34 kata	4 kalimat dan 34 kata	Berhasil	

Program 2

Input	Output Diharapkan	Output Program	Status	Catatan
Tidak ada input (nilai string sudah ditentukan)	K1: Saya tak 'kan menyerah. K2: Ia berkata, "Aku menyayangimu." K3: "Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!" K4: Surat keputusan itu bernomor 62/UN.34/19/2023.	[1] "Saya tak 'kan menyerah." [1] "Ia berkata, "Aku menyayangimu."" [1] ""Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"" [1] "Surat keputusan itu bernomor 62/UN.34/19/2023.	Berhasil	Output sesuai dengan yang diharapkan. Semua karakter khusus (escape character) tertampil dengan benar.

Program 3

Input	Output Diharapkan	Output Program	Status	Catatan
hitung_akar_kuadrat (1, -4, 4)	Persamaan tersebut memiliki akar real kembar: Akar = 2.000	Persamaan tersebut memiliki akar real kembar: Akar = 2.000	Berhasil	Diskriminan = 0
hitung_akar_kuadrat (2, -13, 15)	Persamaan tersebut memiliki akar real: Akar 1 = 5.000 Akar 2 = 1.500	Persamaan tersebut memiliki akar real: Akar 1 = 5.000 Akar 2 = 1.500	Berhasil	Diskriminan > 0
hitung_akar_kuadrat (3, -10, 25)	Persamaan tersebut hanya	Persamaan tersebut	Berhasil	Diskriminan < 0

	memiliki akar-akar imajiner	hanya memiliki akar-akar imajiner		
--	-----------------------------	-----------------------------------	--	--

Program 4

Input	Output Diharapkan	Output Program	Status	Catatan
199210252015032015	Tanggal lahir ASN: 25 Oktober 1992	Tanggal lahir ASN: 25 Oktober 1992	Berhasil	8 digit pertama: 1992 10 25
198540122010011003	Bulan Tidak Valid	Bulan Tidak Valid	Berhasil	8 digit pertama: 1985 40 12 (bulan 40 tidak valid)
199812012023021001	Tanggal lahir ASN: 1 Desember 1998	Tanggal lahir ASN: 1 Desember 1998	Berhasil	8 digit pertama: 1998 12 01

Program 5

Input	Output Diharapkan	Output Program	Status	Catatan
cluster_titik(4, 3, 7)	Jarak ke A: 3.3166 Jarak ke B: 6.7082 Jarak ke C: 9.9499 Titik termasuk Cluster A	Jarak ke A: 3.3166 Jarak ke B: 6.7082 Jarak ke C: 9.9499 Titik termasuk Cluster A	Berhasil	Titik U(4,3,7) paling dekat ke cluster A

cluster_titik(3, -7, -1)	Jarak ke A: 9.4339 Jarak ke B: 12.8062 Jarak ke C: 4.1231 Titik termasuk Cluster C	Jarak ke A: 9.4339 Jarak ke B: 12.8062 Jarak ke C: 4.1231 Titik termasuk Cluster C	Berhasil	Titik U(3,-7,-1) paling dekat ke cluster C
cluster_titik(0.5, 3.5, 2.5)	Jarak ke A: 2.7386 Jarak ke B: 2.9155 Jarak ke C: 6.5192 Titik termasuk Cluster A	Jarak ke A: 2.7386 Jarak ke B: 2.9155 Jarak ke C: 6.5192 Titik termasuk Cluster A	Berhasil	Titik U(0.5,3.5,2.5) paling dekat ke cluster A

Program 6

Input	Output Diharapkan	Output Program	Status	Catatan
interval_konfidensi(0.4, 100, 0.05)	Proporsi Sampel = 0.4 Ukuran Sampel = 100 Alpha = 0.05 Interval Konfidensi = (0.304 , 0.496)	Proporsi Sampel = 0.4 Ukuran Sampel = 100 Alpha = 0.05 Interval Konfidensi = (0.304 , 0.496)	Berhasil	Interval kepercayaan 95% untuk proporsi populasi berada di antara 0.304 dan 0.496
interval_konfidensi(0.6, 250, 0.01)	Error: Alpha harus bernilai 0.05 atau 0.10.	Error: Alpha harus bernilai 0.05 atau 0.10.	Berhasil	Validasi alpha berhasil menangkap

				input 0.01 yang tidak sesuai
interval_konfidensi(1.2, 150, 0.10)	Error: Proporsi harus berada pada rentang 0 sampai 1.	Error: Proporsi harus berada pada rentang 0 sampai 1.	Berhasil	Validasi proporsi berhasil menangkap input 1.2 yang > 1

6.2. Screenshot Hasil Running

1. Program 1

The screenshot shows the RStudio interface. The console displays the following output:

```

> hitung_kata_dan_kalimat("Ketuhanan Yang Maha Esa. Kemanusiaan yang adil dan beradab. Persatuan Indonesia
ia.")
Teks tersebut memuat 3 kalimat dan 11 kata.
> hitung_kata_dan_kalimat("Fungsi adalah blok kode yang diberi nama dan digunakan untuk menjalankan tugas
tertentu. Fungsi dapat menerima input berupa parameter atau argumen, memproses input tersebut, kemudian
menghasilkan output.")
Teks tersebut memuat 2 kalimat dan 27 kata.
> hitung_kata_dan_kalimat("aku berlari menjauh dari hal yang tak lagi utuh. tetes air mata serupa hujan
basahku sepanjang jalan. dalam lelah kuberharap agar tuhan berikan aku lupa. namun itu tak kudapatkan kisah
lain yang ia berikan.")
Teks tersebut memuat 4 kalimat dan 34 kata.

```

The screenshot shows a Python IDE with the following code and output:

```

def hitung_kata_dan_kalimat(teks):
    daftar_kata = teks.split()
    jumlah_kata = len(daftar_kata)

    daftar_kalimat = teks.split(".")
    jumlah_kalimat = 0

    for kalimat in daftar_kalimat:
        if kalimat.strip() != "":
            jumlah_kalimat += 1

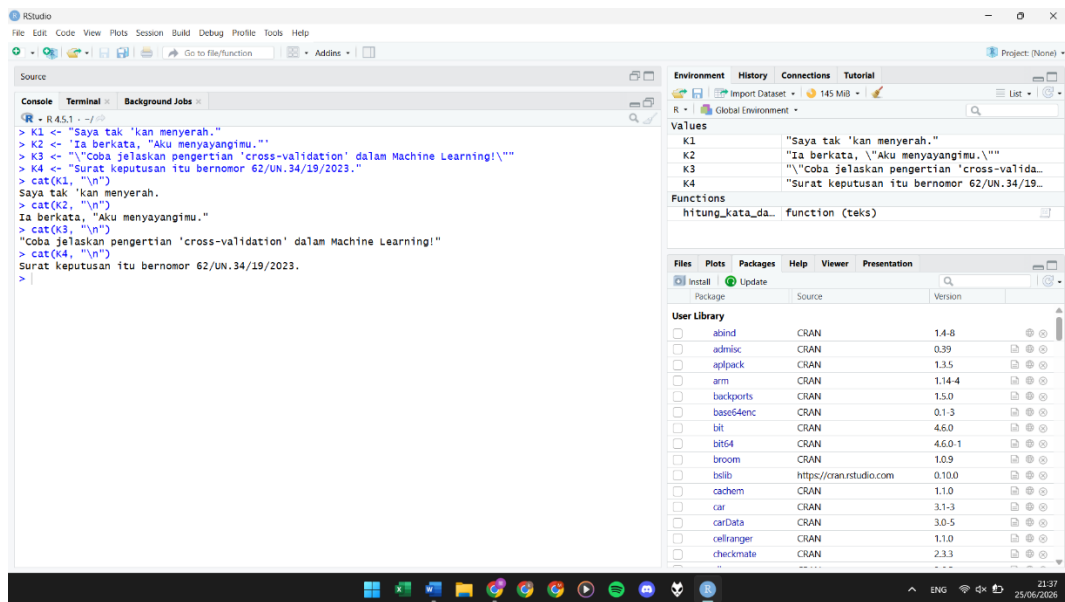
    print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")

# Contoh penanggilan fungsi
hitung_kata_dan_kalimat("Ketuhanan Yang Maha Esa. Kemanusiaan yang adil dan beradab. Persatuan Indonesia.")
hitung_kata_dan_kalimat("Fungsi adalah blok kode yang diberi nama dan digunakan untuk menjalankan tugas tertentu. Fungsi dapat menerima input berupa parameter atau argumen, memproses input tersebut, kemudian menghasilkan output.")
hitung_kata_dan_kalimat("aku berlari menjauh dari hal yang tak lagi utuh. tetes air mata serupa hujan basahku sepanjang jalan. dalam lelah kuberharap agar tuhan berikan aku lupa. namun itu tak kudapatkan kisah lain yang ia berikan.")

Teks tersebut memuat 3 kalimat dan 11 kata.
Teks tersebut memuat 2 kalimat dan 27 kata.
Teks tersebut memuat 4 kalimat dan 34 kata.

```

2. Program 2



```
R - R4.5.1 - ~/R/
> K1 <- "Saya tak 'kan menyerah."
> K2 <- "Ia berkata, 'Aku menyangimu.'"
> K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
> K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."
> cat(K1, "\n")
Saya tak 'kan menyerah.
> cat(K2, "\n")
Ia berkata, "Aku menyangimu."
> cat(K3, "\n")
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
> cat(K4, "\n")
Surat keputusan itu bernomor 62/UN.34/19/2023.
>
```

Environment

Object	Value
K1	"Saya tak 'kan menyerah."
K2	"Ia berkata, 'Aku menyangimu.'" "
K3	"\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\"" "
K4	"Surat keputusan itu bernomor 62/UN.34/19/2023."

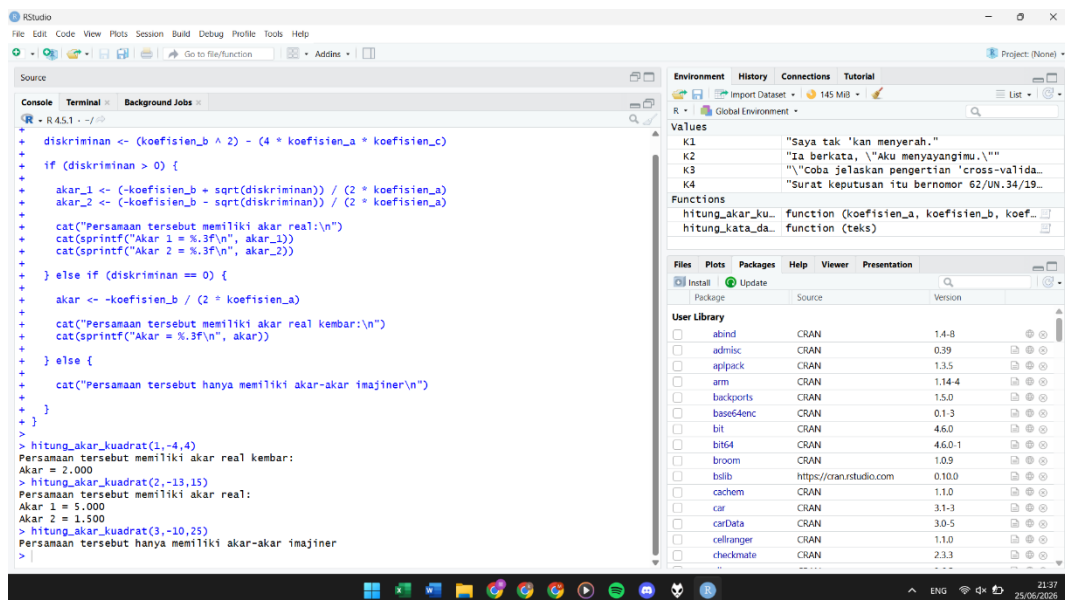
Functions

Function	Value
hitung_kata_da...	function (teks)

User Library

Package	Source	Version
abind	CRAN	1.4-8
admisc	CRAN	0.39
aplpac	CRAN	1.3.5
arm	CRAN	1.14-4
backports	CRAN	1.5.0
base64enc	CRAN	0.1-3
bit	CRAN	4.6.0
bit64	CRAN	4.6.0-1
broom	CRAN	1.0.9
bslib	https://cran.rstudio.com	0.10.0
cachem	CRAN	1.1.0
car	CRAN	3.1-3
carData	CRAN	3.0-5
cellranger	CRAN	1.1.0
checkmate	CRAN	2.3.3

3. Program 3



```
R - R4.5.1 - ~/R/
+ diskriminan <- (koefisien_b ^ 2) - (4 * koefisien_a * koefisien_c)
+ if (diskriminan > 0) {
+   akar_1 <- (-koefisien_b + sqrt(diskriminan)) / (2 * koefisien_a)
+   akar_2 <- (-koefisien_b - sqrt(diskriminan)) / (2 * koefisien_a)
+   cat("Persamaan tersebut memiliki akar real:\n")
+   cat(sprintf("Akar 1 = %.3f\n", akar_1))
+   cat(sprintf("Akar 2 = %.3f\n", akar_2))
+ } else if (diskriminan == 0) {
+   akar <- -koefisien_b / (2 * koefisien_a)
+   cat("Persamaan tersebut memiliki akar real kembar:\n")
+   cat(sprintf("Akar = %.3f\n", akar))
+ } else {
+   cat("Persamaan tersebut hanya memiliki akar-akar imajiner\n")
+ }
+ }
+ }
> hitung_akar_kuadrat(1,-4,4)
Persamaan tersebut memiliki akar real kembar:
Akar = 2.000
> hitung_akar_kuadrat(2,-13,15)
Persamaan tersebut memiliki akar real:
Akar 1 = 5.000
Akar 2 = 1.500
> hitung_akar_kuadrat(3,-10,25)
Persamaan tersebut hanya memiliki akar-akar imajiner
>
```

Environment

Object	Value
K1	"Saya tak 'kan menyerah."
K2	"Ia berkata, 'Aku menyangimu.'" "
K3	"\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\"" "
K4	"Surat keputusan itu bernomor 62/UN.34/19/2023."

Functions

Function	Value
hitung_akar_ku...	function (koefisien_a, koefisien_b, koef...
hitung_kata_da...	function (teks)

User Library

Package	Source	Version
abind	CRAN	1.4-8
admisc	CRAN	0.39
aplpac	CRAN	1.3.5
arm	CRAN	1.14-4
backports	CRAN	1.5.0
base64enc	CRAN	0.1-3
bit	CRAN	4.6.0
bit64	CRAN	4.6.0-1
broom	CRAN	1.0.9
bslib	https://cran.rstudio.com	0.10.0
cachem	CRAN	1.1.0
car	CRAN	3.1-3
carData	CRAN	3.0-5
cellranger	CRAN	1.1.0
checkmate	CRAN	2.3.3

```

import math
def hitung_akar_kuadrat(koefisien_a, koefisien_b, koefisien_c):
    diskriminan = (koefisien_b**2) - (4 * koefisien_a * koefisien_c)
    if diskriminan > 0:
        akar_1 = (-koefisien_b + math.sqrt(diskriminan)) / (2 * koefisien_a)
        akar_2 = (-koefisien_b - math.sqrt(diskriminan)) / (2 * koefisien_a)
        print("Persamaan tersebut memiliki akar real:")
        print("Akar 1 = {akar_1:.3f}"
        print("Akar 2 = {akar_2:.3f}"

    elif diskriminan == 0:
        akar = -koefisien_b / (2 * koefisien_a)
        print("Persamaan tersebut memiliki akar real kembar:")
        print("Akar = {akar:.3f}"

    else:
        print("Persamaan tersebut hanya memiliki akar-akar imajiner")

#contoh pemanggilan fungsi
hitung_akar_kuadrat(1, -4, 4)
hitung_akar_kuadrat(1, -11, 15)
hitung_akar_kuadrat(1, -18, 25)

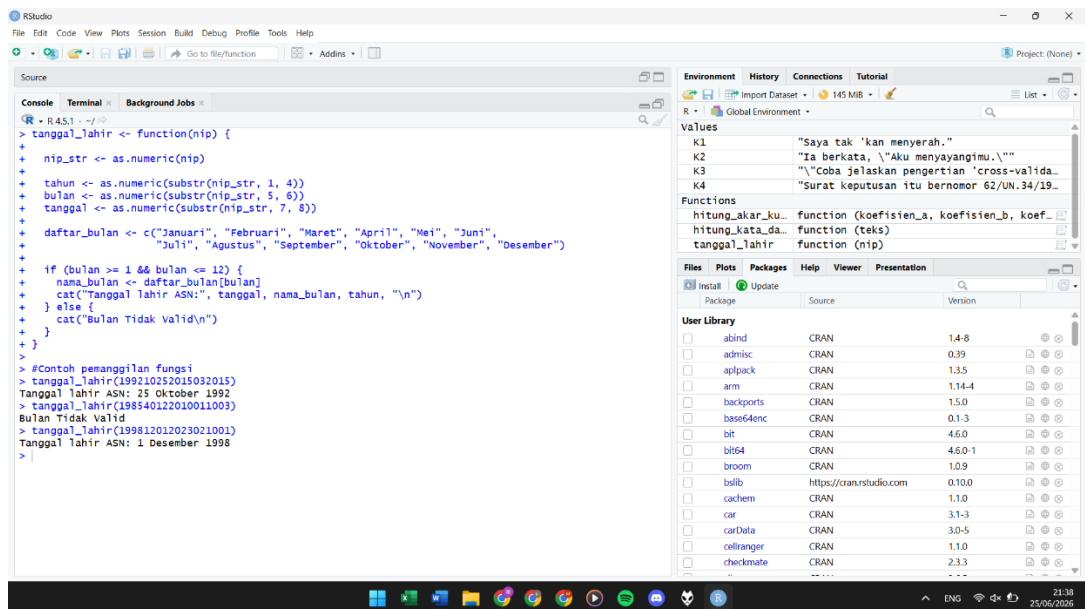
--- Persamaan tersebut memiliki akar real kembar:
Akar = 2.000

Persamaan tersebut memiliki akar real:
Akar 1 = 5.000
Akar 2 = 1.500

Persamaan tersebut hanya memiliki akar-akar imajiner

```

4. Program 4



```

def tanggal_lahir(nip):
    nip_str = str(nip)
    tahun = int(nip_str[0:4])
    bulan = int(nip_str[4:6])
    tanggal = int(nip_str[6:8])

    daftar_bulan = [
        "Januari",
        "Februari",
        "Maret",
        "April",
        "Mei",
        "Juni",
        "Juli",
        "Agustus",
        "September",
        "Oktober",
        "November",
        "Desember"
    ]

    if 1 <= bulan <= 12:
        nama_bulan = daftar_bulan[bulan - 1]
        print("Tanggal lahir ASN: (tanggal, (nama_bulan, tahun)\n")
    else:
        print("Bulan Tidak Valid")

#contoh pemanggilan fungsi
tanggal_lahir(199210252015032015)
tanggal_lahir(198540122010011003)
tanggal_lahir(199812012023021001)

Tanggal lahir ASN: 25 Oktober 1992
Bulan Tidak Valid
Tanggal lahir ASN: 1 Desember 1998

```


5. Program 5

The image displays two screenshots of code environments. The top screenshot is RStudio, showing R code for hierarchical clustering. The bottom screenshot is a Jupyter Notebook, showing the same logic translated into Python using NumPy and SciPy.

RStudio Screenshot:

```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source
> R4.5.1 - /
> cluster_titik <- function(x1, x2, x3) {
+   a <- c(x1, 2, 4)
+   b <- c(-2, 5, 1)
+   c <- c(1, -3, -2)
+   u <- c(x1, x2, x3)
+   da <- sqrt(sum((u - a)^2))
+   db <- sqrt(sum((u - b)^2))
+   dc <- sqrt(sum((u - c)^2))
+   print(paste("Jarak ke A:", round(da, 4)))
+   print(paste("Jarak ke B:", round(db, 4)))
+   print(paste("Jarak ke C:", round(dc, 4)))
+   jarak_min <- min(da, db, dc)
+   cluster_terpilih <- c()
+   if (da == jarak_min) {
+     cluster_terpilih <- c(cluster_terpilih, "cluster A")
+   }
+   if (db == jarak_min) {
+     cluster_terpilih <- c(cluster_terpilih, "cluster B")
+   }
+   if (dc == jarak_min) {
+     cluster_terpilih <- c(cluster_terpilih, "cluster C")
+   }
+   cat("Titik termasuk", paste(cluster_terpilih, collapse = " dan "), "\n\n")
+ }
> cluster_titik(4, 3, 7)
[1] "Jarak ke A: 3.166"
[1] "Jarak ke B: 8.124"
[1] "Jarak ke C: 11.225"
Titik termasuk Cluster A
> cluster_titik(3, 7, -1)
[1] "Jarak ke A: 10.2956"
[1] "Jarak ke B: 13.1529"
[1] "Jarak ke C: 4.3626"
Titik termasuk Cluster C
> cluster_titik(0.5, 3.5, 2.5)
[1] "Jarak ke A: 3.2787"
[1] "Jarak ke B: 3.2787"
[1] "Jarak ke C: 7.9215"
Titik termasuk Cluster A dan Cluster B
```

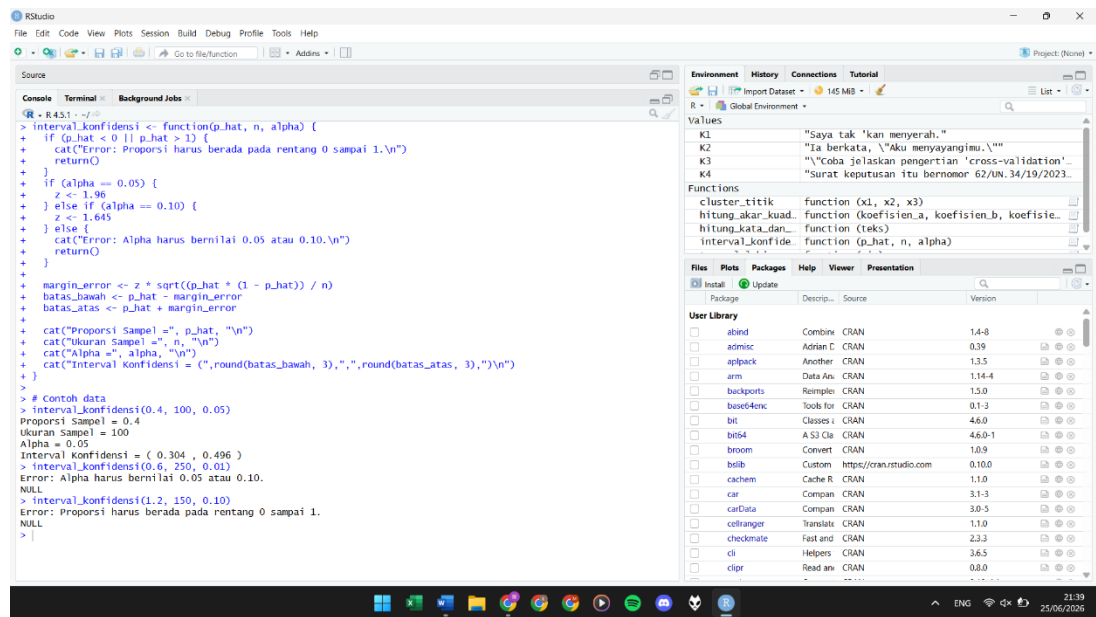
Jupyter Notebook Screenshot:

```
Python 3.6.ipyth
File Edit View Insert Runtime Tools Help
> Import numpy as np
def cluster_titik(x1, x2, x3):
    a = np.array([1, 2, 4])
    b = np.array([-2, 5, 1])
    c = np.array([1, -3, -2])
    u = np.array([x1, x2, x3])
    da = np.sqrt(np.sum((u - a)**2))
    db = np.sqrt(np.sum((u - b)**2))
    dc = np.sqrt(np.sum((u - c)**2))
    print("Jarak ke A: %.4f" % da)
    print("Jarak ke B: %.4f" % db)
    print("Jarak ke C: %.4f" % dc)
    jarak_min = min(da, db, dc)
    cluster_terpilih = []
    if da == jarak_min:
        cluster_terpilih.append("cluster A")
    if db == jarak_min:
        cluster_terpilih.append("cluster B")
    if dc == jarak_min:
        cluster_terpilih.append("cluster C")
    output_cluster = " dan ".join(cluster_terpilih)
    print("Titik termasuk (%s)" % output_cluster)

#Contoh pemanggilan fungsi
cluster_titik(4, 3, 7)
cluster_titik(3, 7, -1)
cluster_titik(0.5, 3.5, 2.5)

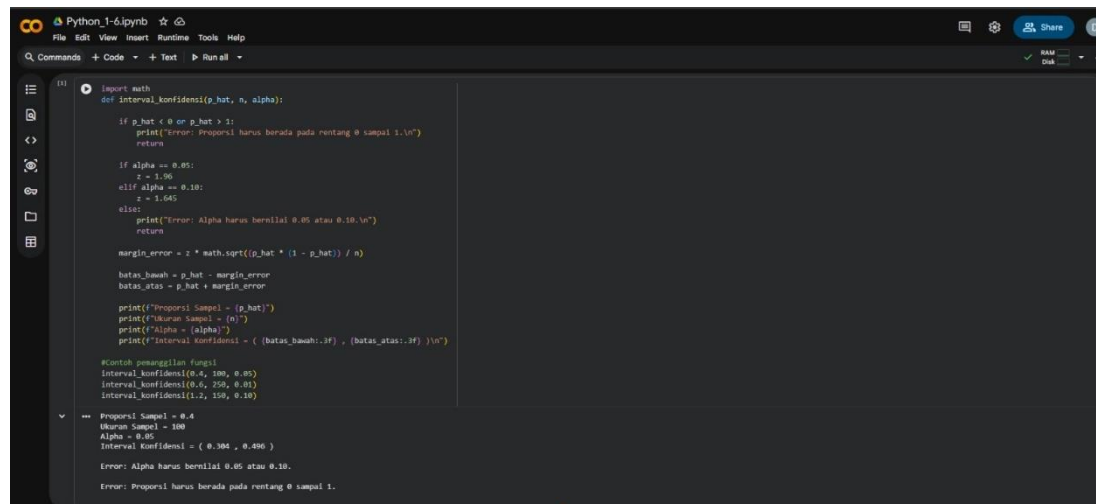
> Jarak ke A: 3.166
> Jarak ke B: 8.124
> Jarak ke C: 11.225
> Titik termasuk Cluster A
> Jarak ke A: 10.2956
> Jarak ke B: 13.1529
> Jarak ke C: 4.3626
> Titik termasuk Cluster C
> Jarak ke A: 3.2787
> Jarak ke B: 3.2787
> Jarak ke C: 7.9215
> Titik termasuk Cluster A dan Cluster B
```

6. Program 6



```
> interval_konfidensi <- function(p_hat, n, alpha) {  
+   if (p_hat < 0 || p_hat > 1) {  
+     cat("Error: Proporsi harus berada pada rentang 0 sampai 1.\n")  
+     return()  
+   }  
+   if (alpha == 0.05) {  
+     z <- 1.96  
+   } else if (alpha == 0.10) {  
+     z <- 1.645  
+   } else {  
+     cat("Error: Alpha harus bernilai 0.05 atau 0.10.\n")  
+     return()  
+   }  
+   margin_error <- sqrt((p_hat * (1 - p_hat)) / n)  
+   batas_bawah <- p_hat - margin_error  
+   batas_atas <- p_hat + margin_error  
+   cat("Proporsi Sampel = ", p_hat, "\n")  
+   cat("Ukuran Sampel = ", n, "\n")  
+   cat("Alpha = ", alpha, "\n")  
+   cat("Interval Konfidensi = (", round(batas_bawah, 3), ", ", round(batas_atas, 3), ")\n")  
+ }  
  
# Contoh data  
> interval_konfidensi(0.4, 100, 0.05)  
Proporsi Sampel = 0.4  
Ukuran Sampel = 100  
Alpha = 0.05  
Interval konfidensi = ( 0.304 , 0.496 )  
> interval_konfidensi(0.6, 250, 0.01)  
Error: Alpha harus bernilai 0.05 atau 0.10.  
NULL  
> interval_konfidensi(1.2, 150, 0.10)  
Error: Proporsi harus berada pada rentang 0 sampai 1.  
NULL  
> |
```

The RStudio interface shows the script in the Source pane and the output in the Console. The Environment pane on the right lists variables K1, K2, K3, and K4 with their respective values.



```
import math  
def interval_konfidensi(p_hat, n, alpha):  
    if p_hat < 0 or p_hat > 1:  
        print("Error: Proporsi harus berada pada rentang 0 sampai 1.\n")  
        return  
    if alpha == 0.05:  
        z = 1.96  
    elif alpha == 0.10:  
        z = 1.645  
    else:  
        print("Error: Alpha harus bernilai 0.05 atau 0.10.\n")  
        return  
    margin_error = z * math.sqrt((p_hat * (1 - p_hat)) / n)  
    batas_bawah = p_hat - margin_error  
    batas_atas = p_hat + margin_error  
    print("Proporsi Sampel = {p_hat}")  
    print("Ukuran Sampel = {n}")  
    print("Alpha = {alpha}")  
    print("Interval Konfidensi = ( {batas_bawah:.3f} , {batas_atas:.3f} )\n")  
  
#Contoh pemanggilan fungsi  
interval_konfidensi(0.4, 100, 0.05)  
interval_konfidensi(0.6, 250, 0.01)  
interval_konfidensi(1.2, 150, 0.10)  
  
Proporsi Sampel = 0.4  
Ukuran Sampel = 100  
Alpha = 0.05  
Interval konfidensi = ( 0.304 , 0.496 )  
  
Error: Alpha harus bernilai 0.05 atau 0.10.  
  
Error: Proporsi harus berada pada rentang 0 sampai 1.
```

The Python IDE shows the script in the editor and the output in the console. The output matches the R version, showing the confidence interval for the first case and error messages for the others.

6.3. Interpretasi Output

Berdasarkan hasil pengujian pada Program 1, program mampu menghitung jumlah kata dan jumlah kalimat dengan benar dari teks yang dimasukkan pengguna. Hasil keluaran pada setiap skenario uji menunjukkan bahwa proses pemisahan teks berdasarkan spasi dan tanda titik berjalan sesuai dengan aturan yang ditetapkan pada soal. Ini membuktikan bahwa logika program sudah tepat selama input mengikuti batasan bahwa tanda titik hanya digunakan sebagai penutup kalimat.

Pada Program 2, output yang dihasilkan sudah sesuai dengan bentuk kalimat yang diminta, termasuk penggunaan tanda petik tunggal, tanda petik ganda, dan simbol khusus lainnya. Perbedaan tampilan antara Python dan R terlihat pada format penulisan output, tetapi isi kalimat tetap sama dan sesuai dengan ketentuan soal. Hal ini menunjukkan bahwa

pengaturan string dan escape character pada kedua bahasa telah berhasil diterapkan dengan benar.

Untuk Program 3, hasil pengujian menunjukkan bahwa program dapat membedakan tiga kondisi utama, yaitu akar real kembar, akar real berbeda, dan akar imajiner. Ketika diskriminan bernilai nol, program menghasilkan satu akar real kembar; ketika diskriminan positif, program menampilkan dua akar real; dan ketika diskriminan negatif, program memberi keterangan bahwa persamaan hanya memiliki akar-akar imajiner. Format output tiga desimal juga sudah muncul dengan baik sesuai ketentuan soal.

Pada Program 4, interpretasi output menunjukkan bahwa program berhasil mengekstrak delapan digit pertama NIP untuk menentukan tahun, bulan, dan tanggal lahir ASN. Selain itu, percabangan untuk menerjemahkan angka bulan ke nama bulan bekerja dengan baik, termasuk saat diuji dengan input yang valid maupun saat muncul bulan tidak valid. Ini berarti validasi dan pemetaan bulan sudah berjalan sesuai rancangan program.

Hasil pengujian Program 5 memperlihatkan bahwa program dapat menghitung jarak titik ke masing-masing pusat cluster dengan benar, lalu menentukan cluster yang paling dekat dengan titik input. Pada setiap skenario uji, output cluster yang dipilih selalu sesuai dengan hasil perbandingan jarak terkecil. Dengan demikian, fungsi perhitungan jarak tiga dimensi dan penentuan cluster sudah berjalan konsisten sesuai tujuan program.

Sementara itu, pada Program 6, output menunjukkan bahwa program berhasil menghitung interval konfidensi proporsi berdasarkan nilai proporsi sampel, ukuran sampel, dan tingkat signifikansi yang diberikan. Saat nilai alpha sesuai ketentuan, program menampilkan batas bawah dan batas atas interval dengan benar; sedangkan pada input yang tidak valid, seperti proporsi di luar rentang 0 sampai 1 atau alpha yang tidak sesuai, program menampilkan pesan kesalahan. Ini menunjukkan bahwa validasi input dan proses perhitungan interval konfidensi telah berjalan sesuai spesifikasi soal.

Secara keseluruhan, seluruh hasil pengujian membuktikan bahwa keenam program telah menghasilkan output yang sesuai dengan kebutuhan masing-masing soal. Output yang ditampilkan tidak hanya benar secara logika, tetapi juga sudah memenuhi ketentuan format, validasi input, dan perbedaan implementasi antara Python dan R.

BAB VII

PENUTUP

7.1. Kesimpulan

Melalui *project* akhir ini, kelompok kami berhasil menyusun dan membandingkan enam program menggunakan dua bahasa pemrograman, yaitu Python dan R. Kami menyadari bahwa sebelum mulai menulis kode (*coding*), pembuatan alur logika melalui *pseudocode* dan *flowchart* sangatlah penting agar program tidak mudah *error*.

Dari hasil pengerjaan, kami juga melihat bahwa setiap bahasa memiliki kelebihan masing-masing. Python terasa lebih mudah dan fleksibel untuk mengolah teks (*string*), sedangkan R lebih praktis digunakan untuk perhitungan angka dan rumus-rumus statistik. Secara keseluruhan, *project* ini sangat membantu kami dalam memahami dasar-dasar logika komputasi.

7.2. Kendala yang Dihadapi

Selama pengerjaan *project*, terdapat beberapa kendala yang kami hadapi. Salah satu kendala utama muncul pada Program 4, yaitu menentukan tanggal lahir dari NIP ASN, terutama ketika harus menangani tanggal pada bulan Februari yang dapat berbeda antara tahun kabisat dan bukan tahun kabisat. Karena soal hanya meminta pengambilan tanggal dari delapan digit pertama NIP, validasi lanjutan seperti pengecekan apakah 29 Februari valid terhadap tahun lahir belum sepenuhnya dibahas dalam program dasar. Selain itu, pada beberapa soal kami juga perlu berhati-hati dalam penulisan string, tanda kutip, dan escape character agar output Python dan R sesuai dengan format yang diminta. Kendala lain adalah memastikan logika program di Python dan R tetap setara meskipun sintaks yang digunakan berbeda.

7.3. Saran Pengembangan

Untuk pengembangan selanjutnya, program dapat dibuat lebih kompleks dan lebih fleksibel. Khusus pada Program 4, program dapat dikembangkan agar tidak hanya menampilkan tanggal lahir dari NIP, tetapi juga memvalidasi apakah tanggal tersebut benar secara kalender, termasuk pengecekan bulan Februari pada tahun kabisat dan jumlah hari pada setiap bulan. Dengan begitu, program akan lebih akurat dan mampu menangani input yang lebih beragam. Selain itu, pengembangan lain dapat berupa penambahan fitur antarmuka pengguna, penyederhanaan output, atau perluasan skenario uji agar program

lebih robust dan mudah digunakan. Untuk soal 5 dan 6, program juga dapat dikembangkan dengan penanganan kasus khusus yang lebih lengkap, misalnya jika dua cluster memiliki jarak yang sama atau jika input statistik berada di luar kondisi yang umum.

LAMPIRAN

Link GitHub : <https://github.com/danishprasetya-pc/TUGAS-PROJECT-KELOMPOK-D>

Link video demo:

<https://drive.google.com/file/d/1PXx4Pg4Dr6FyhLlORFv8rkPuasb8JSTV/view?usp=sharing>

Link penggunaan AI : <https://chatgpt.com/share/6a23c592-1af8-83ec-ab11-512b1e5de661>